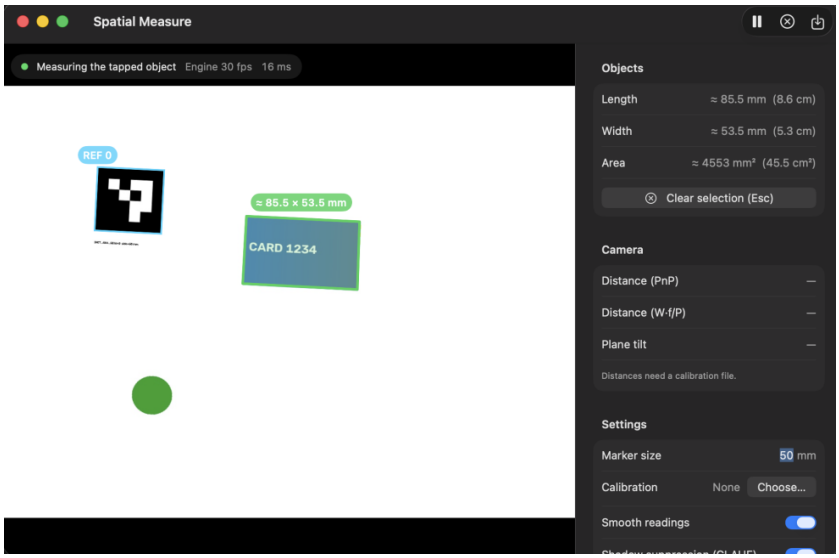
PROJECT REPORT

Real-Time Spatial Distance and Object Dimension Estimation

Using Perspective Geometry, Planar Homography and Learned Segmentation



Submitted by

Joel Prasad	URK24CS7019
Nevin Reji	URK24CS7004
Prince Amos	URK24CS7024

Dept. of Computer Science and Engineering (AIML)
Karunya Institute of Technology and Sciences
Coimbatore, India

October 2026

Abstract

This project estimates the real size of an object, in millimetres, from the video of one ordinary camera. The user clicks on the object in the live picture; the program outlines it, reports its length, width and area, and keeps following it as it moves. The scale comes from a printed ArUco marker of known size, whose four corners define a planar homography from the tilted camera view to a top-down view measured in millimetres. The outline of the clicked object is obtained from a pretrained segmentation model (MobileSAM), with a colour and edge method as a fallback, and the smallest rotated rectangle around the outline gives the length and width.

Three measuring modes are provided: a table mode in which the camera may move so that objects larger than one frame can be measured, a hand mode in which the object is held next to the marker, and a no-marker mode in which the scale comes from a distance entered by the user and the result is labelled as an estimate.

On rendered scenes with known dimensions, an 85.6 × 53.98 mm card was recovered to within about 1.2 % at distances up to 900 mm and within about 3 % at 1.5 m, and a 400 × 80 mm board was measured by panning. On six real video sequences from DAVIS 2017, the pretrained model kept the outline on the clicked object better than the colour and edge method (mean overlap J = 0.40 against 0.05). A trial with a physical camera has not yet been recorded, so no real-world accuracy is claimed.

Keywords: monocular measurement, planar homography, ArUco marker, camera calibration, object segmentation, MobileSAM, DAVIS.

Table of Contents

Chapter 1: Introduction 1
1.1 Background 1
1.2 Problem Statement 1
1.3 Motivation 2
1.4 Objectives 2
1.5 Chapter-wise Summary 3
Chapter 2: Literature Review 4
2.1 Introduction 4
2.2 Review of Existing Systems 4
2.3 Research Gaps 6
Chapter 3: System Analysis and Requirements 7
3.1 Existing System 7
3.2 Proposed System 7
3.3 Functional Requirements 8
3.4 Non-Functional Requirements 8
3.5 Hardware and Software Requirements 8
Chapter 4: System Design and Methodology 9
4.1 System Architecture 9
4.2 Block Diagram 9
4.3 System Workflow 10
4.4 Algorithms Used 11
4.5 Methodology 13
Chapter 5: Implementation 15
5.1 Development Environment 15
5.2 Dataset or Input Description 15
5.3 Module Description 15
5.4 Implementation Steps 16
5.5 Code Explanation 17
Chapter 6: Results and Discussion 18
6.1 Experimental Setup 18
6.2 Output Screenshots 19

6.3 Results Obtained 21
6.4 Performance Evaluation 23
6.5 Discussion of Results 25
Chapter 7: Conclusion and Future Scope 26
7.1 Conclusion 26
7.2 Future Work 26
References 27
Appendices 28
Appendix A: Source Code 28
Appendix B: Additional Output Screenshots 45
Appendix C: GitHub Repository Link 47
Appendix D: Video Demonstration Link 47

Chapter 1: Introduction

1.1 Background

Measuring the size of an object without touching it is useful in site inspection, quality control, logistics and robotic handling. Calipers and tapes need contact. Depth sensors such as stereo cameras, time-of-flight modules and LiDAR give metric depth directly, but they add hardware cost and need their own calibration. A single ordinary camera is cheap and already present in every laptop and phone, but one image does not determine size, for three reasons:

- **Perspective foreshortening:** an object that is farther from the lens covers fewer pixels, so pixel counts alone say nothing about length.
- **Angular projection bias:** if the optical axis is not perpendicular to the surface, a rectangle appears as an irregular four-sided shape and equal lengths no longer have equal pixel lengths.
- **Radial and tangential lens distortion:** real lenses bend straight lines, so the mapping from pixels to length is not linear across the frame.

This project, called Spatial Measure, addresses these three problems in software. Each frame is first corrected for lens distortion using a camera calibration. A printed ArUco marker of known side length is then detected, and its four corners define a planar homography that maps the tilted view to a top-down view in which one pixel corresponds to a known number of millimetres. The user clicks on an object; a segmentation step finds the pixels of that object, and the smallest rotated rectangle around them gives its length and width. The cycle Capture → Undistort → Detect marker → Homography → Segment → Measure → Smooth → Display is repeated for every frame, so the readings follow the object as it moves.

The project starts from the pipeline described in the source paper [1] and extends it in three directions: objects larger than the camera view can be measured by following the table surface while the camera moves, objects held in the hand can be followed, and a mode without a marker takes its scale from a distance entered by the user.

1.2 Problem Statement

Given the live video of one uncalibrated or calibrated consumer camera, the problem is to report the length, width and area, in millimetres, of an object chosen by the user with a single click, together with the distance from the camera to the measuring surface, in real time. The method must remove perspective, tilt and lens distortion; it must measure only the chosen object and not its background or the hand that holds it; it must cope with objects larger than

one camera frame; and it must state clearly when a result is only an estimate because no physical reference is visible.

1.3 Motivation

The idea started from site work. Officers who visit a site cannot carry large measuring and scaling equipment, but they always have a camera with them. A tool that gives an approximate size from a click on the camera view would let them make quick estimates without special hardware. The estimate does not have to be survey-grade, but it has to be reasonable, fast and clear about its limits.

Cost is a second reason. The source paper [1] compares a monocular camera, costing under 20 US dollars, with a stereo pair at 150 to 300 US dollars, a time-of-flight module at 100 to 250 US dollars and LiDAR at 500 to 5000 US dollars. Deep networks can find an object in an image but cannot give its size in millimetres without a physical reference, because depth predicted from a single image is ambiguous in scale [2, 3].

The third reason comes from how the first version behaved. It handled only flat objects lying next to a marker and measured every object in view. In practice an object is held in the hand, is sometimes larger than the camera view, and sometimes no marker is available. The colour and edge method used at first also failed on cluttered backgrounds (Section 6.4.2), which led to the use of a trained segmentation model.

1.4 Objectives

The aim is a real-time, single-camera system that reports the size of the object the user clicks, with an accuracy of about one millimetre at close range when a marker is used, and a clearly marked estimate otherwise. The specific objectives are to:

1. derive the model from image coordinates (u, v) to metric plane coordinates (X, Y) using the pinhole model, lens distortion and planar homography;
2. obtain the millimetre scale from an ArUco marker of known printed size;
3. measure the object that the user clicks, outline it, and follow it as it moves, in real time;
4. measure objects larger than one camera frame by following the table while the camera moves;
5. support an object held in the hand, with and without a marker, without including the fingers;
6. make the outline follow only the clicked object, and measure this on the DAVIS 2017 dataset;
7. provide a native macOS application with a live preview; and

8. define the tests and metrics, keeping rendered-scene tests separate from physical-camera trials.

1.5 Chapter-wise Summary

Chapter 2 reviews the literature on calibration, single-view metrology, fiducial markers, plane tracking and learned segmentation, and states the research gap. **Chapter 3** analyses the existing system and lists the proposed system and its functional, non-functional, hardware and software requirements. **Chapter 4** presents the architecture, block diagram, workflow, the algorithms with their equations, and the methodology of the three measuring modes. **Chapter 5** describes the development environment, the inputs and datasets, the modules, the implementation steps and the key code. **Chapter 6** gives the experimental setup, output screenshots, results, performance evaluation and discussion. **Chapter 7** concludes and lists future work. The appendices contain the source code, additional screenshots and the repository and video links.

Chapter 2: Literature Review

2.1 Introduction

The related work was grouped into six areas that correspond to the stages of the pipeline: camera geometry and calibration, measurement from a single image, fiducial markers and plane pose, plane tracking, classical outline methods, and learned segmentation and depth. Twenty works are reviewed, including the source paper. The methods and limitations are described as recognised in the literature; the experiments of the original authors were not repeated.

2.2 Review of Existing Systems

Table 2.1 gives the method, the limitation and the relevance of each work. No single work covers the whole task: each supplies one part of the pipeline.

Table 2.1. Comparison of the reviewed works

Work	Methodology	Limitations	Relevance to this project
Brown (1971) [4]	Plumb-line close-range calibration; radial and decentring distortion model.	Controlled photogrammetric set-up; offline.	Origin of the distortion terms in Equation (4.3).
Tsai (1987) [5]	Two-stage calibration using a radial alignment constraint.	Needs accurate target geometry; sensitive to noise.	Made calibration practical with ordinary cameras.
Heikkilä and Silvén (1997) [6]	Four-step calibration with explicit and implicit distortion correction.	More complex than a closed-form method.	Alternative calibration route.
Zhang (2000) [7]	Intrinsics from several views of a flat pattern, then non-linear refinement.	Pattern must stay flat; accuracy depends on the views.	Method used by calibrate.py.
Criminisi et al. (2000) [8]	Single view metrology: lengths on planes from one known reference.	Needs a reference and parallel structure; manual steps.	Same principle: one known length gives metric scale.
Hartley and Zisserman (2004) [9]	Projective geometry; homography by DLT and SVD, RANSAC.	Theory text; not a measurement system.	Basis of Equation (4.5) and Equation (4.6).
Kato and Billinghurst (1999) [10]	ARToolKit: square black-border markers, pose from the corners.	Sensitive to lighting and occlusion.	Early square-marker system.
Fiala (2005) [11]	ARTag: digitally coded markers with error correction.	Fixed marker set; weaker for small or blurred markers.	Error-correcting marker coding.
Olson (2011) [12]	AprilTag: lexicode tags, gradient-based line detection.	More computation; needs clear space around the tag.	Alternative marker family.
Garrido-Jurado et al. (2014) [13]	ArUco: generated dictionaries with large inter-marker distance.	Fails with strong glare, heavy blur or hidden border.	Marker family used here (DICT_4X4_50).
Romero-Ramirez et al. (2018) [14]	Faster detection of square fiducial markers.	Whole marker border must be visible.	Supports the real-time requirement.
Collins and Bartoli (2014) [15]	IPPE: closed-form plane pose from a homography.	Planar targets only; two-fold ambiguity near frontal views.	Used for tilt and distance.
Rublee et al. (2011) [16]	ORB: oriented FAST keypoints with rotated BRIEF descriptors.	Weak on textureless surfaces; error accumulates when chained.	Plane tracking in table mode.
Fischler and Bolles (1981) [17]	RANSAC: robust model fitting with outliers.	Random; depends on the inlier threshold.	Robust homography between views.

Work	Methodology	Limitations	Relevance to this project
Eigen et al. (2014) [2]	CNN that predicts a depth map from one image.	Scale is ambiguous; not millimetre-accurate.	Why depth networks are not used for size.
Ranftl et al. (2022) [3]	Mixed-dataset training for robust relative depth.	Relative depth only; no metric scale.	As above.
Kirillov et al. (2023) [18]	Segment Anything: mask from a click, trained on SA-1B.	Masks, not size; large model; may return a part.	Basis of the Smart outline.
Zhang et al. (2023) [19]	MobileSAM: smaller, faster Segment Anything.	Part-or-whole ambiguity; weaker on thin objects.	Model used in the application.
Perazzi et al.; Pont-Tuset et al. [20, 21]	DAVIS: video object segmentation benchmark.	A benchmark; scenes harder than a desk.	Used in Section 6.4.2.
Source paper [1]	Undistortion, ArUco homography, Canny contours, minAreaRect, 1080p overlay.	Flat objects facing the camera; marker always in view.	Baseline pipeline extended here.

2.2.1 Camera Geometry and Calibration

The pinhole model and the lens distortion terms were described in photogrammetry by Brown [4]. Tsai [5] and Heikkilä and Silvén [6] made calibration practical for computer vision, and Zhang [7] showed how the camera matrix and distortion coefficients can be obtained from several views of a flat checkerboard. This method is available in OpenCV [22] and is used by the calibration script. Hartley and Zisserman [9] explain the projective geometry behind homography estimation with the Direct Linear Transform (DLT).

2.2.2 Measuring from a Single Image

Criminisi, Reid and Zisserman [8] showed that lengths on a plane can be recovered from one image if one reference length is known. The present system uses the same idea with a simpler reference: a printed marker whose four corners give the homography directly. The limit is also known from that work: it measures only what lies on the reference plane.

2.2.3 Fiducial Markers and Plane Pose

ARToolKit [10], ARTag [11] and AprilTag [12] are square markers that can be detected and located in an image. ArUco [13] generates dictionaries with error correction, and Romero-Ramirez et al. [14] made detection faster. For a square marker the plane pose can be obtained with EPnP [23] or with the plane-specific IPPE method [15], which is used here. These works supply the reference that gives scale, but none of them measures objects.

2.2.4 Tracking a Plane

ORB features [16] are fast to compute and match, and matches filtered by a ratio test [24] and RANSAC [17] give the homography between two views of the same plane. Chaining such homographies keeps track of the table after the marker has left the view. The known weakness is accumulated error, so the system returns to the exact marker homography whenever the marker is seen again.

2.2.5 Classical Outline Methods

The source pipeline uses CLAHE [25], Canny edges [26], Otsu thresholding [27] and contour following [28]. These work on a plain background but depend on strong, closed edges, so they fail on cluttered scenes; Section 6.4.2 measures this on real video.

2.2.6 Learned Segmentation and Depth

Depth networks [2, 3] predict depth from one image, but the scale is relative, so they cannot by themselves give millimetres. Segment Anything [18] was trained on 11 million images with 1.1 billion masks and returns the mask of the object under a click; MobileSAM [19] is a smaller version fast enough for live use. The DAVIS benchmark [20, 21] provides video with ground-truth masks.

2.3 Research Gaps

The geometry needed to turn pixels into millimetres on a plane is well established, learned segmentation now gives the outline of an arbitrary object from one click, and depth networks do not replace a physical reference. The works also share limits: markers must stay flat and visible, plane tracking drifts when chained, and segmentation models may return only part of an object.

Within the works reviewed, none combines (a) a single ordinary camera, (b) selection of an arbitrary object by a click with a learned outline, (c) metric scale from a printed marker, (d) extension beyond the frame by following the surface, and (e) an explicit estimate mode with no marker whose label states that the result depends on an assumed distance. This is the gap the project addresses; it is a statement about the reviewed works, not a claim that no such system exists.

Chapter 3: System Analysis and Requirements

3.1 Existing System

Non-contact size measurement is done today in three ways. Hand tools (tape, ruler, caliper) are accurate but need contact and an operator at the object. Depth sensors (stereo, time-of-flight, LiDAR) give metric depth but cost 100 to 5000 US dollars and need their own calibration [1]. Single-camera pipelines such as the source paper [1] undistort the frame, detect an ArUco marker, compute a homography and fit rectangles to Canny contours.

The single-camera pipeline of the source paper has these limitations, which were confirmed while re-implementing it:

- it measures every contour in view instead of the object the user wants;
- the listing computes the homography but does not warp the frame with it, and maps the scene into a marker-sized square;
- adaptive thresholding followed by Canny gives double edges, and contours sit 1 to 2 px outside the true edge, so every size is too large;
- the distance $D = Wf/P$ is valid only when the marker faces the camera;
- the marker must always be in view, the object must lie on the table, and the edge-based outline fails on cluttered backgrounds.

3.2 Proposed System

Spatial Measure keeps the geometric core of the source pipeline (calibration, ArUco homography, rotated rectangle) and changes how the object is chosen and followed. The user clicks the object; a learned segmentation model outlines it; the outline is converted to millimetres through the marker homography; and the reading is smoothed and followed over time. Three modes extend the system to large objects, to objects in the hand, and to scenes without a marker. Table 3.1 lists the features.

Table 3.1. Main features of the proposed system

Feature	Purpose
Click to measure	The clicked object is outlined; length, width and area follow it while it stays in view.
Table mode with scanning	The plane is tracked after the marker leaves the view, so objects larger than one frame can be measured.
Hand mode with marker	The marker is held beside the object; the object can move and turn; fingers are excluded.
Hand mode without marker	Scale from an entered distance and the focal length; shown as an estimate.
Smart outline	MobileSAM outlines the clicked object; a colour and edge method is the fallback.
Distance and tilt	Camera-to-marker distance and plane tilt when a calibration file is loaded.

3.3 Functional Requirements

Table 3.2. Functional requirements

ID	Requirement
FR-1	Capture live video from the built-in or a USB camera and show it at the camera frame rate.
FR-2	Calibrate the camera from checkerboard images and save K and the distortion coefficients.
FR-3	Detect the ArUco reference marker (DICT_4X4_50) and compute the plane homography.
FR-4	Let the user select an object with one click and outline only that object.
FR-5	Report length, width and area in millimetres and the spacing between objects.
FR-6	Report the camera-to-marker distance and the plane tilt, and warn above 65°.
FR-7	Follow the selected object across frames and smooth the readings.
FR-8	Measure objects larger than the frame by scanning, and show whether the object is complete.
FR-9	Provide hand modes with and without a marker; label marker-less readings as estimates.
FR-10	Log readings to CSV and compute accuracy tables against known sizes.

3.4 Non-Functional Requirements

Table 3.3. Non-functional requirements

Quality	Requirement
Performance	Preview at the full camera rate (30 FPS); classical engine under 33 ms per frame; AI outline about 70 ms per frame on Apple silicon; late frames dropped, never queued.
Accuracy	About 1 mm at close range with a marker; reprojection error below 0.5 px after calibration.
Usability	One click to measure; Esc to clear; settings in a sidebar; estimates marked with ≈ or “at least”.
Reliability	Falls back to the classical outline if the model is unavailable; single bad frames rejected.
Privacy	All frames processed locally; no image sent to an external service.
Maintainability	Python core library separated from the Swift interface; automated tests need no camera.
Portability	Core library and OpenCV-window version run on any OS with Python; the native app needs macOS.

3.5 Hardware and Software Requirements

Table 3.4. Hardware and software requirements

Item	Minimum	Used in this project
Computer	Mac with Apple silicon, 8 GB RAM	Apple M4 Pro, macOS 26
Camera	720p webcam	Built-in 1080p camera, 30 FPS
Printer	Any printer at 100 % scale	50 mm ArUco marker; 9 × 6 checkerboard, 25 mm squares
Language	Python 3.10+, Swift 5	Python 3.13.5; SwiftUI with Xcode CLT
Libraries	OpenCV ≥ 4.8, NumPy ≥ 1.24, Pillow	OpenCV 5.0.0, NumPy 2.5.3
Optional	ultralytics ≥ 8.3, PyTorch	MobileSAM weights (about 40 MB)
Testing	pytest ≥ 7	11 automated tests

Chapter 4: System Design and Methodology

4.1 System Architecture

Spatial Measure is a pipeline repeated for every video frame (Figure 4.1). The top row converts the camera frame into a metric description of the plane: the frame is undistorted, the ArUco marker is detected, and its four corners give the plane homography. The middle row decides which pixels belong to the object, using the click, the segmentation step and a hand-removal step. The third row selects the mode, and the bottom row measures, smooths and displays the result.

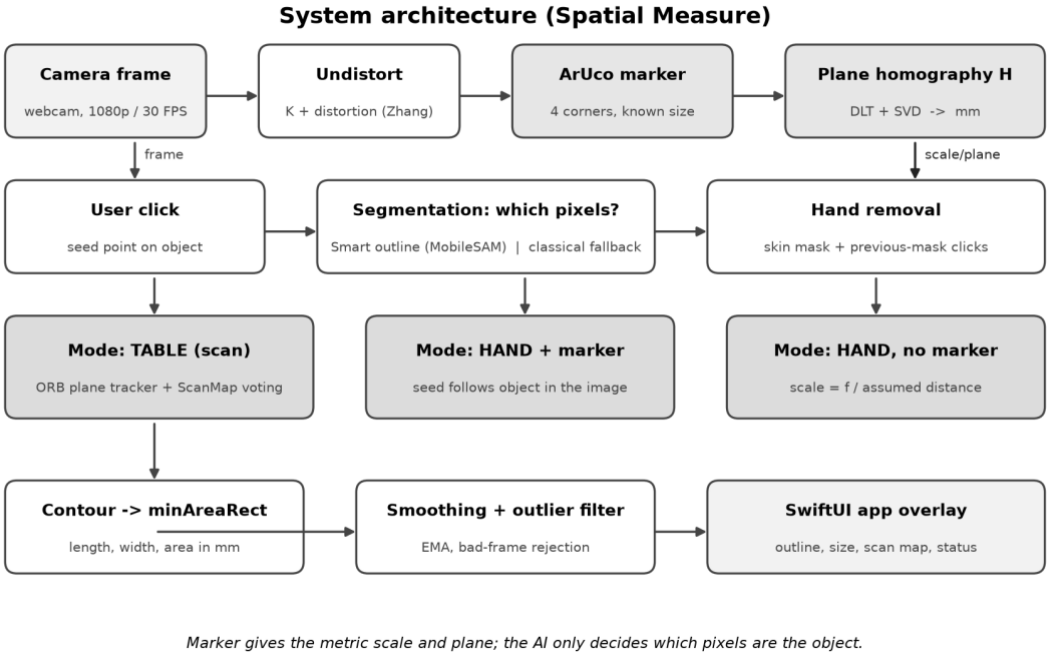


Figure 4.1. System architecture of Spatial Measure (conceptual design of the implemented pipeline).

4.2 Block Diagram

Figure 4.2 shows the software blocks. The macOS application captures the camera and draws the interface; it sends frames to the Python engine over standard input and receives results over standard output. The engine uses the estimator for geometry and measurement, the segmenter for the learned outline and the scan module for plane tracking. Calibration is done once, offline, and its file is loaded by the estimator.

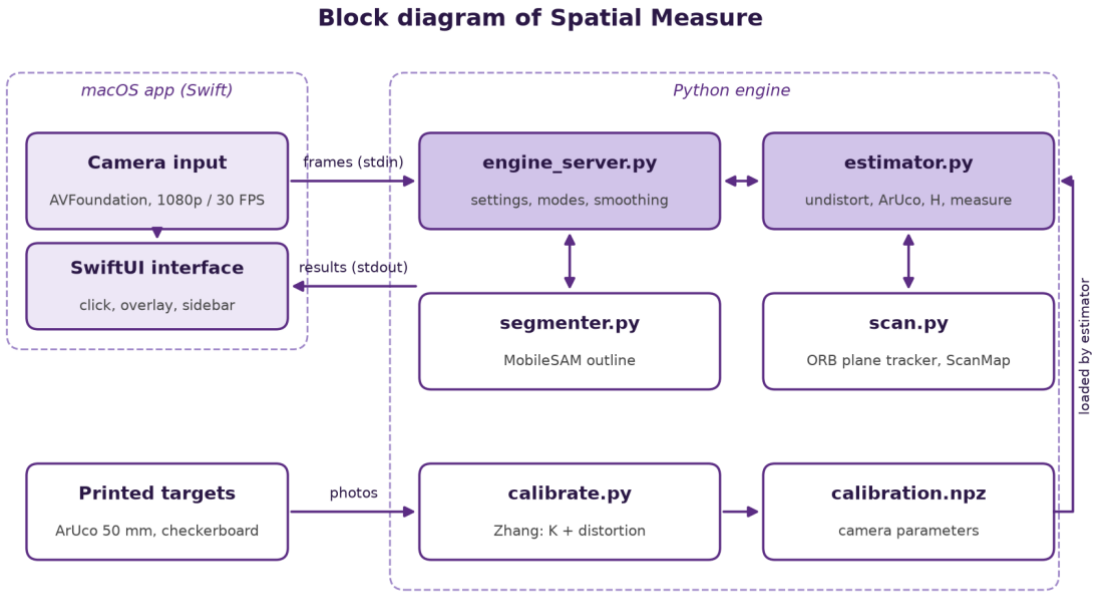


Figure 4.2. Block diagram of the software modules and the data that flows between them.

4.3 System Workflow

Table 4.1 summarises the input, processing and output of each stage of the per-frame workflow. From the user's side the workflow is:

1. print the marker and checkerboard at 100 % scale and measure the printed marker;
2. calibrate the camera once (optional for sizes, needed for distance);
3. place the marker flat beside the object, or hold it beside the object in hand mode;
4. click the object; read length, width, area, distance and tilt; press Esc to clear.

Table 4.1. Stages of the per-frame measurement pipeline

Stage	Input	Processing	Output
1. Capture and undistort	Camera frame; K and distortion	Remap with cached tables (30 FPS)	Undistorted frame
2. Marker detection	Greyscale frame	ArUco detection (DICT_4X4_50)	Four corners and marker id
3. Plane homography	Corners; marker side W_m	Homography image → top-down canvas at ρ px/mm	H, canvas, ρ
4. Object selection	Click (u, v); frame	MobileSAM or colour/edge fallback; hand removal	Object mask and contour
5. Measurement	Contour; H; ρ	Minimum-area rotated rectangle; px → mm	Length, width, area
6. Distance and tilt	Corners; K	Similar triangles; IPPE pose	D, D_pnp, tilt θ
7. Smoothing	Readings per frame	Exponential average; outlier gate	Stable readings
8. Display	Readings; outline	SwiftUI overlay and sidebar	On-screen result

4.4 Algorithms Used

4.4.1 Pinhole Camera and Plane Homography

A world point (X, Y, Z) is mapped to a pixel (u, v) through the rotation $\boldsymbol{R}$ and translation $\boldsymbol{t}$ of the camera and the camera matrix $\boldsymbol{K}$ with focal lengths f_x, f_y and principal point (c_x, c_y); s is the depth along the optical axis (Equation (4.1)).

$$s\begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix} [\boldsymbol{r}_1 \quad \boldsymbol{r}_2 \quad \boldsymbol{r}_3 \quad \boldsymbol{t}] \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix} \tag{4.1}$$

All measured objects lie on the marker plane $Z = 0$, so $\boldsymbol{r}_3$ drops out and the mapping reduces to a 3×3 homography $\boldsymbol{H}$ (Equation (4.2)). $\boldsymbol{H}$ has eight degrees of freedom, so four point correspondences determine it.

$$s\,\tilde{\boldsymbol{u}} = \boldsymbol{H}\,\tilde{\boldsymbol{X}}_p, \qquad \boldsymbol{H} = \boldsymbol{K}[\boldsymbol{r}_1 \quad \boldsymbol{r}_2 \quad \boldsymbol{t}] \tag{4.2}$$

4.4.2 Lens Distortion and Zhang Calibration

With normalised coordinates x, y and $r^2 = x^2 + y^2$, the distorted coordinates follow the radial (k_1, k_2, k_3) and tangential (p_1, p_2) model of Brown [4] (Equation (4.3)).

$$\begin{aligned} x_d &= x(1 + k_1 r^2 + k_2 r^4 + k_3 r^6) + 2p_1 xy + p_2(r^2 + 2x^2) \\ y_d &= y(1 + k_1 r^2 + k_2 r^4 + k_3 r^6) + p_1(r^2 + 2y^2) + 2p_2 xy \end{aligned} \tag{4.3}$$

$\boldsymbol{K}$ and the five coefficients are found once with Zhang's method [7] from 15 to 30 views of a 9×6 checkerboard. Quality is judged by the RMS reprojection error (Equation (4.4)); the target is below 0.5 px. At run time each frame is undistorted with precomputed remap tables.

$$\varepsilon_{\text{rms}} = \sqrt{\frac{1}{N}\sum_{i=1}^{N} \|\boldsymbol{p}_i - \hat{\boldsymbol{p}}_i\|^2} \tag{4.4}$$

4.4.3 DLT Homography from the ArUco Marker

The four detected corners (u_i, v_i) of a marker of side W_m (50 mm) correspond to known plane points (X_i, Y_i). Each gives two linear equations in the nine entries $\boldsymbol{h}$ of $\boldsymbol{G} = \boldsymbol{H}^{-1}$ [9] (Equation (4.5)).

$$\begin{bmatrix} -u_i & -v_i & -1 & 0 & 0 & 0 & u_i X_i & v_i X_i & X_i \\ 0 & 0 & 0 & -u_i & -v_i & -1 & u_i Y_i & v_i Y_i & Y_i \end{bmatrix} \boldsymbol{h} = \boldsymbol{0} \tag{4.5}$$

Stacking four correspondences gives $\boldsymbol{Ah} = \mathbf{0}$, solved by the right singular vector of the smallest singular value (Equation (4.6)).

$$\boldsymbol{A} = \boldsymbol{U\Sigma V}^T, \qquad \boldsymbol{h} = \boldsymbol{v}_9 \tag{4.6}$$

The plane is mapped to a top-down canvas of ρ px/mm (4 by default) with the marker corners at the points of Equation (4.7); a canvas point is converted to millimetres by Equation (4.8). For $W_m = 50$ mm and $\rho = 4$, $s = 200$ px, and an 85.6 × 53.98 mm card covers about 342 × 216 px.

$$(0,0),\ (s,0),\ (s,s),\ (0,s), \qquad s = \rho\, W_m \tag{4.7}$$

$$X = \frac{x_c - x_0}{\rho}, \qquad Y = \frac{y_c - y_0}{\rho} \tag{4.8}$$

4.4.4 Distance and Tilt: Similar Triangles and IPPE

For a frontal marker of apparent mean side P pixels, the distance follows from similar triangles (Equation (4.9)), the formula of the source paper [1].

$$D = \frac{W_m\, f}{P}, \qquad f = \frac{f_x + f_y}{2}$$
$$P = \frac{1}{4}\sum_{k=1}^{4} \|\boldsymbol{p}_{k+1} - \boldsymbol{p}_k\| \tag{4.9}$$

The second estimate solves the full pose with IPPE [15]; the distance is $\| \boldsymbol{t} \|$ and the tilt follows from R_{33} (Equation (4.10)). The application warns when $\theta > 65°$. Example: with $f = 1140.75$ px and $P = 95$ px, $D = 50 \times 1140.75/95 = 600.4$ mm.

$$\theta = \arccos|R_{33}| \tag{4.10}$$

4.4.5 Object Outline: MobileSAM and Classical Fallback

Smart outline. MobileSAM [19], a distilled Segment Anything [18], receives the click as a positive point and returns the object mask. Up to three more points are chosen deep inside the previous mask to keep the outline on the same object. In the hand modes, skin-coloured pixels (YCrCb, Cr 135–175, Cb 85–125) are removed so that fingers are not included.

Classical fallback. The frame is warped to the canvas, CLAHE [25] is applied to the L channel of LAB, Canny edges [26] are computed and closed, and the region around the click bounded by the edges is flood-filled. The Canny contour sits 1–2 px outside the true edge

(0.5–1.0 mm at 4 px/mm), so it is re-located at the local Otsu threshold [27] (Equation (4.11)).

$$\sigma_B^2(t) = \omega_0(t)\,\omega_1(t)[\mu_0(t) - \mu_1(t)]^2$$
$$t^* = \underset{t}{\operatorname{argmax}}\ \sigma_B^2(t) \tag{4.11}$$

4.4.6 Dimension Computation with minAreaRect

`minAreaRect` returns the sides w_p, h_p of the smallest rotated rectangle in canvas pixels; length, width and area follow from Equation (4.12), where A_p is the contour area. Candidates with a short side below 2 mm or an area below 20 mm² are rejected.

$$L = \frac{\max(w_p, h_p)}{\rho}, \quad W = \frac{\min(w_p, h_p)}{\rho}, \quad A = \frac{A_p}{\rho^2} \tag{4.12}$$

4.4.7 Smoothing and Outlier Rejection

The reported size s_t is an exponential moving average of the per-frame measurement m_t with $\alpha = 0.25$; in the hand modes a frame that differs by more than 35 % is skipped, for at most four frames (Equation (4.13)).

$$s_t = \begin{cases} s_{t-1} & \text{if } |m_t - s_{t-1}| > 0.35\, s_{t-1} \\ s_{t-1} + \alpha(m_t - s_{t-1}) & \text{otherwise} \end{cases} \tag{4.13}$$

4.4.8 Plane Tracking with ORB and RANSAC

When the marker leaves the view, ORB features [16] (1200 features, 6 levels) are matched against the nearest stored keyframes with a 0.75 ratio test [24], and a homography $\boldsymbol{H}_{t\to k}$ is fitted with RANSAC [17] (3 px). A match needs at least 18 inliers, an inlier ratio of 0.35 and a scale between 0.4 and 2.5. The plane mapping is the chain of Equation (4.14).

$$\boldsymbol{M}_t = \boldsymbol{M}_k\, \boldsymbol{H}_{t\to k} \tag{4.14}$$

4.5 Methodology

The work followed an iterative method. The source-paper pipeline was first re-implemented and tested on rendered scenes with known geometry; each defect found (missing warp, double edges, contour bias, frontal-only distance) was corrected and covered by a test; the click-to-measure interface and the three modes were then added, each with its own rendered-scene test; finally the outline stage was evaluated on real video (DAVIS). The three modes differ only in how the outline is converted to millimetres.

4.5.1 Table Mode and Scanning

The marker lies flat beside the object. While it is visible, the exact homography is used and keyframes are stored (at most 40). After it leaves the view, the plane is followed with ORB (Equation (4.14)). Each frame's outline is drawn into a 4000 × 4000 cell map at 2 cells/mm; a cell counts only after repeated observation. The label reads "at least L × W" while the outline touches unseen area, and "about L × W" once the whole object has been seen.

4.5.2 Hand Mode with a Marker

The user holds the marker next to the object at the same distance from the camera. The click for the next frame is a point deep inside the mask just found, so the selection follows the object; fingers are removed and single bad frames are filtered by Equation (4.13). An object closer to the lens than the marker reads too large.

4.5.3 Hand Mode without a Marker

The focal length comes from the calibration or from the horizontal field of view φ (65° by default) and image width w (Equation (4.15)). At distance Z one millimetre covers ρ pixels (Equation (4.16)), and an outline of P_L pixels has length L (Equation (4.17)).

$$f = \frac{w/2}{\tan(\varphi/2)} \tag{4.15}$$

$$\rho = \frac{f}{Z} \tag{4.16}$$

$$L = \frac{P_L}{\rho} = \frac{P_L Z}{f} \tag{4.17}$$

Differentiating gives Equation (4.18): the relative error in size equals the relative error in the entered distance. With $f = 1142.38$ px, a card at 400 mm entered as 500 mm reads $L = 107.0$ mm, 25 % too large. This is why these readings are labelled as estimates.

$$\frac{\partial L}{\partial Z} = \frac{L}{Z} \Rightarrow \frac{\delta L}{L} = \frac{\delta Z}{Z} \tag{4.18}$$

4.5.4 Operational Considerations

The marker must be printed at 100 % scale and its side measured and entered, because an error in W_m scales every result. The marker and object must lie flat on one surface; glare, hard shadows and motion blur break contours. Distance readings need a calibration file; sizes do not, because the homography handles perspective.

Chapter 5: Implementation

5.1 Development Environment

The core library (`spatial_measure/`) is written in Python 3.13 with OpenCV [22] 5.0 and NumPy 2.5, developed in Visual Studio Code with a virtual environment (`.venv`). The measurement engine (`engine_server.py`) wraps the library and exchanges messages with the application over standard input and output. The application (`macapp/`) is a native macOS program in SwiftUI with AVFoundation, built with the Xcode command line tools. The camera is shown through `AVCaptureVideoPreviewLayer` at the full frame rate; frames go to the engine only when it is idle, and late frames are dropped. MobileSAM runs through `ultralytics` and PyTorch on the Apple GPU (MPS). Tests use pytest; development and timing were done on an Apple M4 Pro.

5.2 Dataset or Input Description

Table 5.1. Inputs and datasets

Input	Description
Live video	1920 × 1080 frames at 30 FPS from the Mac camera (BGRA, sent as raw bytes to the engine).
Reference marker	ArUco DICT_4X4_50, id 0, printed at 50 mm side by `generate_targets.py`.
Calibration images	15–30 views of a 9 × 6 checkerboard with 25 mm squares; output `calibration.npz`.
Rendered scenes	500 × 350 mm plane with marker, 85.6 × 53.98 mm card and 50 mm disc, projected through K of the source paper at known tilt and distance (ground truth for tests).
DAVIS 2017	Real video with per-frame object masks [21]; 6 sequences × 30 frames used.
MobileSAM weights	Pretrained on SA-1B (11 M images, 1.1 B masks); downloaded once to `models/`.

5.3 Module Description

Table 5.2. Modules of the system

Module	Role
calibrate.py	Zhang's method; saves K and distortion to calibration.npz.
estimator.py	Undistortion, marker detection, homography, canvas, outline, rectangle fit, distance and tilt.
segmenter.py	MobileSAM through ultralytics; click plus extra points from the previous mask.
scan.py	PlaneTracker (marker or chained ORB homography) and ScanMap (outline accumulated on the plane).
engine_server.py	Settings, frame loop, smoothing, outlier gate, modes and the message protocol.
measure.py	OpenCV-window version for webcam, video or image, with CSV logging.
macapp/	Live preview, click handling, overlay, sidebar with readings and scan map.
evaluate.py, evaluate_tracking.py	Accuracy tables from logs; DAVIS click-and-follow evaluation.

Module	Role
tests/	Eleven automated tests on rendered scenes with known ground truth.

5.4 Implementation Steps

The source-paper listing was corrected and extended as summarised in Table 5.3. The system is then built and run as follows.

Table 5.3. Changes made to the source-paper pipeline

Change	Where	Reason
Rectification and canvas	estimator.compute_homography	The listing computes H but never warps with it; the marker now defines the mm scale and the canvas covers the frame.
Undistortion with cached maps	estimator.undistort	Needed for 30 FPS; keeping all valid pixels avoids shifting the principal point.
CLAHE on the L channel	estimator.preprocess	Adaptive threshold followed by Canny gave double edges.
Otsu edge refinement	estimator.refine_contour	Canny contours sit 1 to 2 px outside the true edge.
Click to select and measure	estimator.select_object; app	Measure the clicked object instead of every object in view.
Large objects (scan mode)	scan.py	Follow the table with ORB after the marker leaves the view.
Hand modes	estimator.process_nomarker; engine	Measure a moving object, with or without a marker.
Trained outline	segmenter.py	The classical outline fails on natural backgrounds.
PnP distance and tilt	estimator.estimate_distance	D = W f / P is valid only for a frontal marker; IPPE also gives tilt.

1. Create the environment and install the requirements:

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

2. Run the automated tests (no camera needed):

```
python -m pytest -q
```

3. Print the targets at 100 % and check the marker with a ruler:

```
python generate_targets.py
```

4. Calibrate (SPACE to capture, C to finish):

```
python calibrate.py --camera 0
```

5. Measure in an OpenCV window, or build the app:

```
python measure.py --source 0 \
  --calib calibration.npz --marker-mm 50
./macapp/build.sh
open "build/Spatial Measure.app"
```

6. Evaluate the outline on DAVIS:

```
python evaluate_tracking.py datasets/davis \
  --backend ai --sequences 6 --frames 30
```

5.5 Code Explanation

Homography to a millimetre canvas. The marker corners are mapped to a square of side $s = \rho W_m$; a translation then makes the canvas cover the whole frame.

```
s = self.marker_length_mm * ppm
dst = np.array([[0,0],[s,0],[s,s],[0,s]],
               np.float32)
H = cv2.getPerspectiveTransform(
        marker_corners, dst)
return T @ H, size, marker_poly, ppm, warnings
```

Edge map. CLAHE on the L channel reduces shadows; blur, Canny and a morphological closing give one closed outline per object.

```
gray = self._clahe.apply(cv2.cvtColor(
        warped, cv2.COLOR_BGR2LAB)[:, :, 0])
blurred = cv2.GaussianBlur(gray, (7, 7), 0)
edges = cv2.Canny(blurred, *self.canny_thresholds)
kernel = cv2.getStructuringElement(
        cv2.MORPH_RECT, (5, 5))
edges = cv2.morphologyEx(edges, cv2.MORPH_CLOSE,
        kernel, iterations=2)
```

Otsu refinement. The neighbourhood of each contour is thresholded at Otsu's level and the contour is replaced if the new area is 80–100 % of the old one, removing the outward bias.

```
_, binary = cv2.threshold(roi, 0, 255,
        cv2.THRESH_BINARY + cv2.THRESH_OTSU)
ratio = cv2.contourArea(best) / \
        max(cv2.contourArea(cnt), 1.0)
if not 0.8 <= ratio <= 1.0:
    return cnt
```

Measurement. Each contour gets a rotated rectangle; the marker and slivers are skipped, and pixels become millimetres by dividing by ρ.

```
(cx, cy), (w_px, h_px), angle = cv2.minAreaRect(cnt)
if min(w_px, h_px) < 2 * ppm:   # shadows, seams
    continue
length_mm = max(w_px, h_px) / ppm
width_mm  = min(w_px, h_px) / ppm
```

Distance and tilt. Similar triangles from the mean marker side, and the IPPE pose for any tilt.

```
d_similar = self.marker_length_mm * f / sides.mean()
ok, rvec, tvec = cv2.solvePnP(obj, marker_corners,
        K, np.zeros(5),
        flags=cv2.SOLVEPNP_IPPE_SQUARE)
R, _ = cv2.Rodrigues(rvec)
tilt = math.degrees(math.acos(abs(R[2, 2])))
```

Learned outline. The click and the extra points from the previous mask are passed to MobileSAM as positive prompts; the first mask is returned.

```
pts = [list(point)] + [list(p) for p in extra_points]
res = self.model.predict(frame_bgr, points=[pts],
        labels=[[1] * len(pts)], device=self.device)
m = res[0].masks.data[0].cpu().numpy()
```

Chapter 6: Results and Discussion

6.1 Experimental Setup

Two kinds of evidence exist. The first is eleven automated tests on rendered scenes, in which a flat scene with known dimensions is projected through a known camera pose and the output is compared with the known values; all eleven pass. The second is an evaluation of the click-and-follow outline on six real DAVIS 2017 sequences. A trial with a physical camera and caliper-measured objects has not yet been recorded, so no real-world accuracy is claimed.

The rendered scenes use the camera matrix of the source paper [1]: 1920 × 1080 px, $f_x = 1142.38$, $f_y = 1139.12$, $c_x = 961.04$, $c_y = 538.22$ px. A 500 × 350 mm canvas holds a 50 mm marker, an 85.6 × 53.98 mm card rotated by 20° and a 50 mm disc. Timings were measured on an Apple M4 Pro. Table 6.1 lists the automated tests and Table 6.2 the physical trials still to be done.

Table 6.1. Automated tests on rendered scenes (all passing)

ID	Test	Scenario	Pass criterion	Result
T-01	Dimensions and distance (3 cases)	Card and disc, marker at 600 mm, tilt 0°, 25°, 45°	Within 1.0 mm; PnP distance within 2 %; tilt within 2°	Pass
T-02	No calibration	Tilt 30°, no calibration file	Card within 1.0 mm; no distance	Pass
T-03	No marker in view	Blank frame	No marker and no measurement	Pass
T-04	Click on card and disc	Click on each object	Card within 2.5 mm; disc within 1.5 mm	Pass
T-05	Selection tracked; marker rejected	Repeated frames; click on marker and empty space	Repeats within 0.5 mm; not accepted	Pass
T-06	Object larger than the frame	400 × 80 mm board panned in a 200 mm view	Within 12 × 6 mm; reported complete	Pass
T-07	Hand with marker	Card moved and turned, 30 frames, finger over a corner	Never lost; every length within 3 mm	Pass
T-08	No marker, distance sensitivity	Correct distance and 25 % too large	Within 2.5 mm; ratio 1.25 ± 0.03	Pass
T-09	Smart outline	Tilted scene with MobileSAM	Card within 4 mm	Pass

Table 6.2. Physical trials to be carried out (not yet done)

ID	Trial	Procedure	Metric	Status
PT-01	Calibration quality	20–30 checkerboard views with the real camera	ε_rms < 0.5 px	Planned
PT-02	Static accuracy	5 caliper-measured objects, 10 readings each at 300, 600, 900 mm	Error (mm, %); SD	Planned
PT-03	Tilt sweep	Repeat PT-02 at 0°, 25°, 45°, 60°	Error vs. tilt	Planned
PT-04	Hand with marker	Marker and object at equal depth, moved for 30 s	Error; frames lost	Planned

ID	Trial	Procedure	Metric	Status
PT-05	No-marker sensitivity	Distance entered 20 % low, correct, 20 % high	Error vs. distance	Planned
PT-06	Large object	Scan a 600 mm board on a textured table	Error; drift	Planned
PT-07	Lighting and glare	Dim light, side light, glossy surface	Failure rate	Planned

6.2 Output Screenshots

Figure 6.1 is the output of the pipeline on a rendered scene tilted by 45°: the marker is outlined in blue, each object in green with its size, and the yellow line gives the spacing. Figure 6.2 to Figure 6.4 show the application fed with a rendered scene; they show interface behaviour and are not measurements of a real scene.

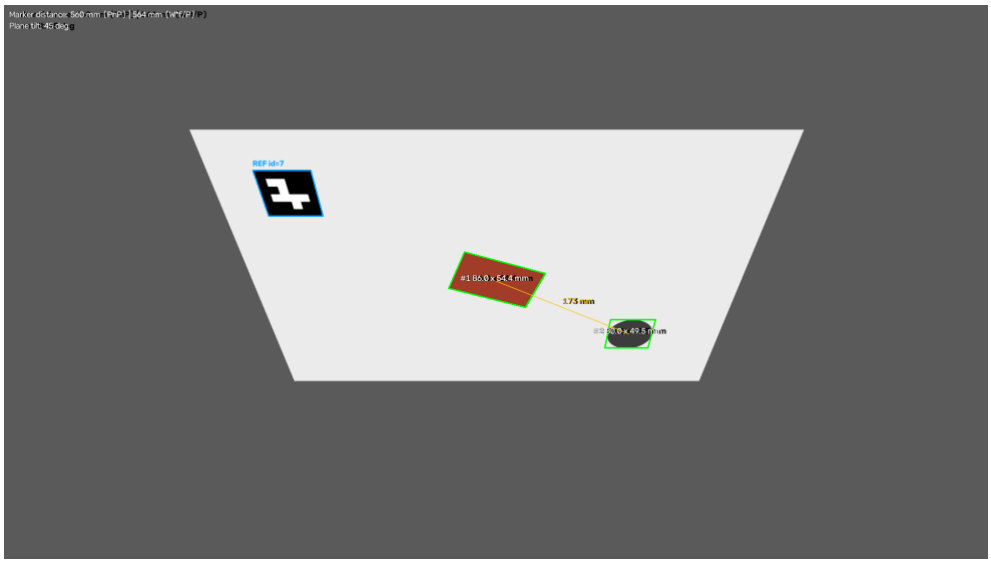


Figure 6.1. Pipeline output, plane tilted 45°, marker at 600 mm (rendered scene). Card 86.0 × 54.4 mm (true 85.6 × 53.98); disc 50.0 × 49.5 mm (true 50).

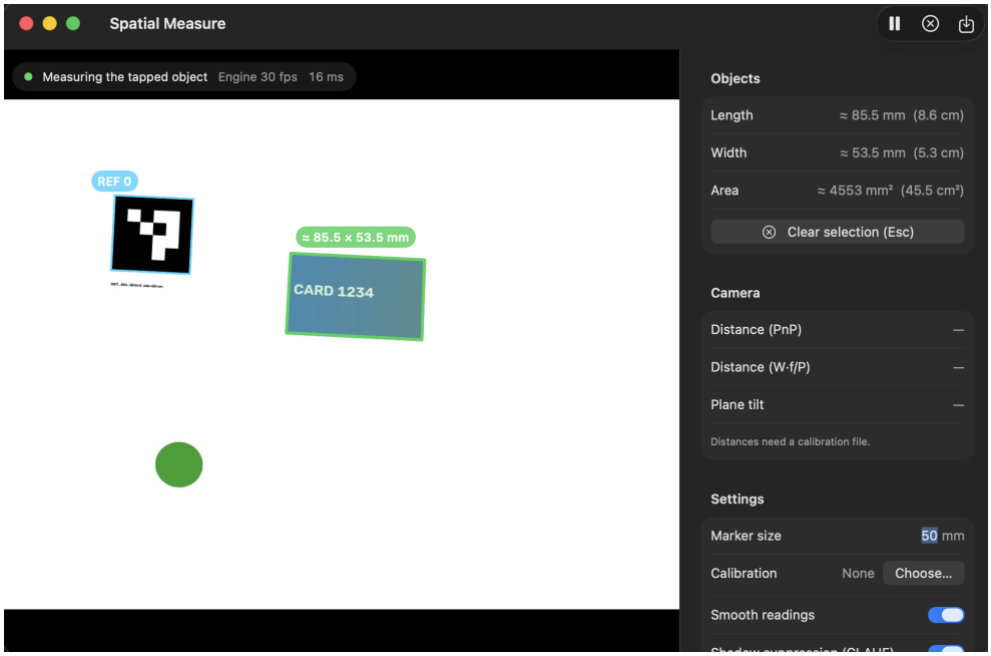


Figure 6.2. Application, tap to measure: the card reads ≈ 85.5 × 53.5 mm, area ≈ 4553 mm²; engine 30 FPS, 16 ms.

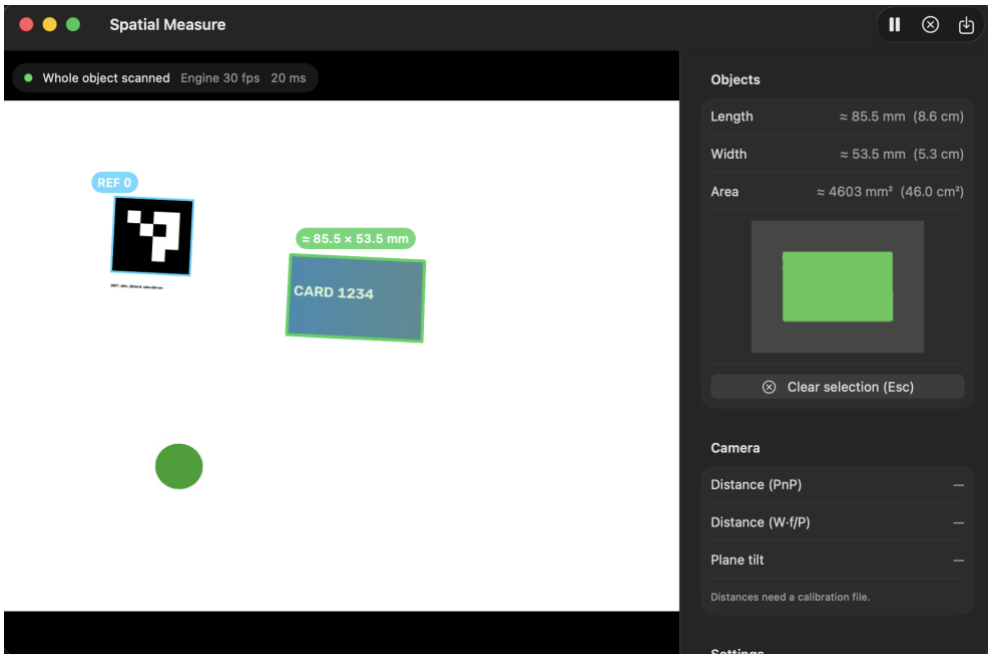


Figure 6.3. Application in scan mode: “Whole object scanned”, with the scan map in the sidebar.

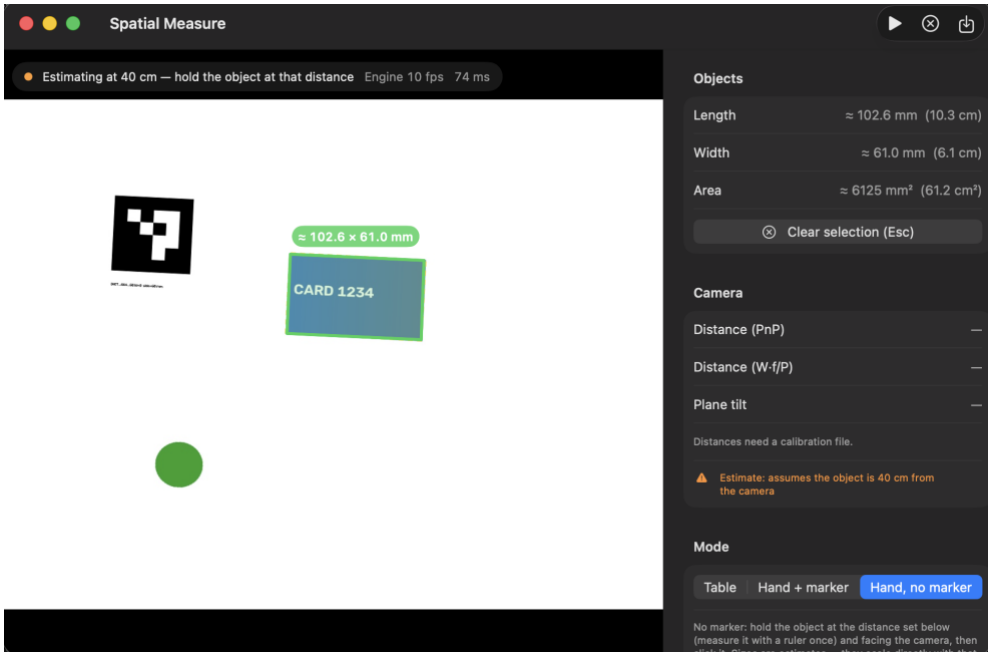


Figure 6.4. Application, hand mode without a marker: the reading is labelled as an estimate assuming 40 cm.

In Figure 6.4 the reading (≈ 102.6 × 61.0 mm) differs from the true card (85.6 × 54.0 mm), which shows that this mode depends on the assumed distance and field of view.

6.3 Results Obtained

6.3.1 Tilt and Distance

Table 6.3 gives the rendered scene at a nominal 600 mm and four tilts; “True Z” is the depth of the marker centre, which Equation (4.9) estimates, and the PnP error is against the true Euclidean distance. Table 6.4 gives a frontal marker at five distances.

Table 6.3. Rendered scene, nominal 600 mm, tilt sweep

Tilt (°)	True Z (mm)	W·f/P (mm)	Error (%)	PnP (mm)	Error (%)	Card L × W (mm)	L error (%)
0	600.0	601.1	0.19	638.7	0.19	86.22 × 54.40	0.72
25	553.5	556.4	0.52	593.2	0.19	86.23 × 54.47	0.74
45	522.2	564.2	8.04	560.2	0.13	86.02 × 54.42	0.49
60	504.7	601.9	19.25	541.2	0.16	86.63 × 54.80	1.20

Table 6.4. Rendered scene, frontal marker, distance sweep

True Z (mm)	W·f/P (mm)	Error (%)	PnP error (%)	Card L × W (mm)	L error (%)	W error (%)
300	300.5	0.16	0.17	85.94 × 54.29	0.40	0.57
600	601.1	0.19	0.19	86.22 × 54.40	0.72	0.77
900	905.0	0.55	0.54	86.33 × 54.58	0.85	1.11
1200	1210.2	0.85	0.70	87.95 × 55.13	2.75	2.14
1500	1508.8	0.59	0.56	88.19 × 55.30	3.03	2.45

The PnP distance stays within 0.2 % at every tilt and the tilt within 0.1°. The similar-triangles distance is within 0.5 % for a near-frontal marker but 8 % too large at 45° and 19 % at 60°, because foreshortening shortens the marker sides. The card size depends on the homography, not on D, and stays within 1.2 % at every tilt; its error is always positive by 0.5–1 mm, matching the contour bias of Section 4.4.5.

6.3.2 Click, Hand and No-Marker Tests

Table 6.5. Recorded values of the remaining tests

Test	True size	Recorded	Note
Click on card (T-04)	85.6 × 54.0 mm	85.5 × 53.5 mm	Also when clicking on printed text
Click on disc (T-04)	Ø 30 mm	30.0 × 29.8 mm	
Panning (T-06)	400 × 80 mm	401 × 81 mm	Marker out of view after 3 frames
Hand, moving (T-07)	85.6 × 54.0 mm	≈ 86.2 × 54.2 mm	30 frames
No marker, 400 mm (T-08)	85.6 × 54.0 mm	85.9 × 54.2 mm	FOV 65°
No marker, 500 mm (T-08)	85.6 × 54.0 mm	107.3 × 67.7 mm	Equation (4.17) predicts 107.0
Smart outline (T-09)	85.6 × 54.0 mm	86.0 × 54.2 mm	70–90 ms per frame

6.3.3 Large Objects and the No-Marker Mode

In the panning test (Figure 6.5) a 400 × 80 mm board was rendered on a textured table and a 200 mm view moved along it. The estimated length grew frame by frame to 401 mm and the object was reported complete only after both ends had been seen. Figure 6.6 shows that the no-marker estimate is proportional to the entered distance, as Equation (4.18) predicts.

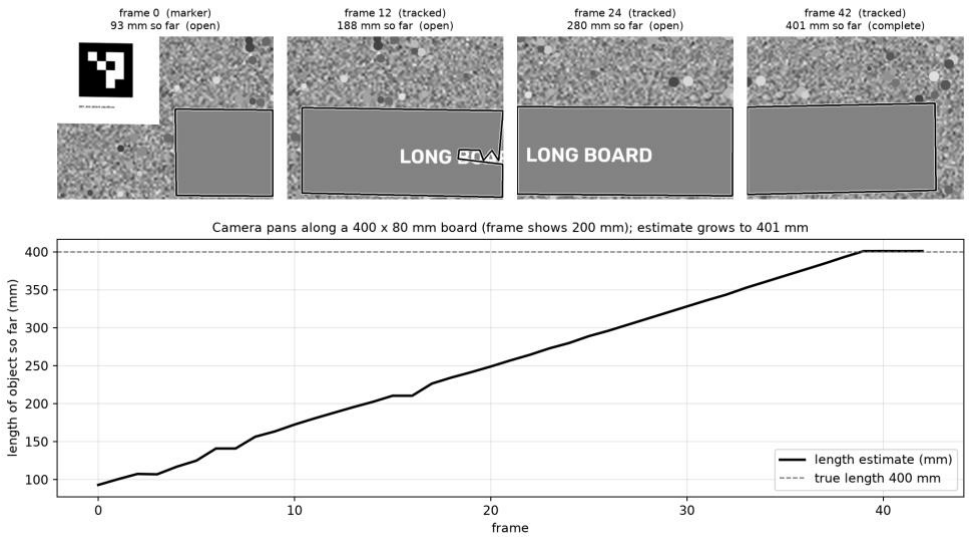


Figure 6.5. Panning test (simulation). Top: frames 0, 12, 24, 42 with the outline. Bottom: estimated length against frame.

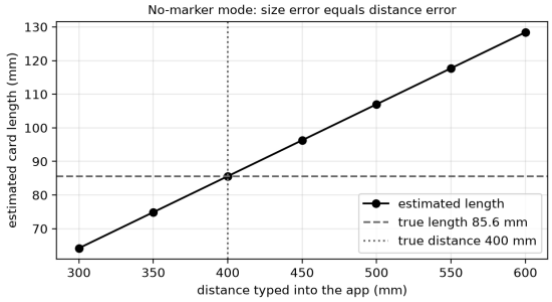


Figure 6.6. Estimated length against entered distance, no-marker mode (simulation; true distance 400 mm).

6.4 Performance Evaluation

Size accuracy is the absolute and percentage error of an estimate $\hat{x}$ against the true x (Equation (6.1)). For outlines, the predicted mask $\mathcal{P}$ is compared with the ground truth $\mathcal{G}$ by the overlap J [20] (Equation (6.2)) and the on-object share (Equation (6.3)); "tracked" is the share of frames with $J \geq 0.5$.

$$e = |\hat{x} - x|, \qquad e_{\%} = \frac{|\hat{x} - x|}{x} \times 100 \tag{6.1}$$

$$J = \frac{|\mathcal{P} \cap \mathcal{G}|}{|\mathcal{P} \cup \mathcal{G}|} \tag{6.2}$$

$$\mathrm{OnObject} = \frac{|\mathcal{P} \cap \mathcal{G}|}{|\mathcal{P}|} \tag{6.3}$$

6.4.1 Measurement Accuracy

The size error is below 1.2 % up to 900 mm and rises to about 3 % at 1.2–1.5 m (Figure 6.7). Beyond 900 mm the card covers fewer than 255 px, so one pixel is more than 0.3 % of its length and edge location dominates the error.

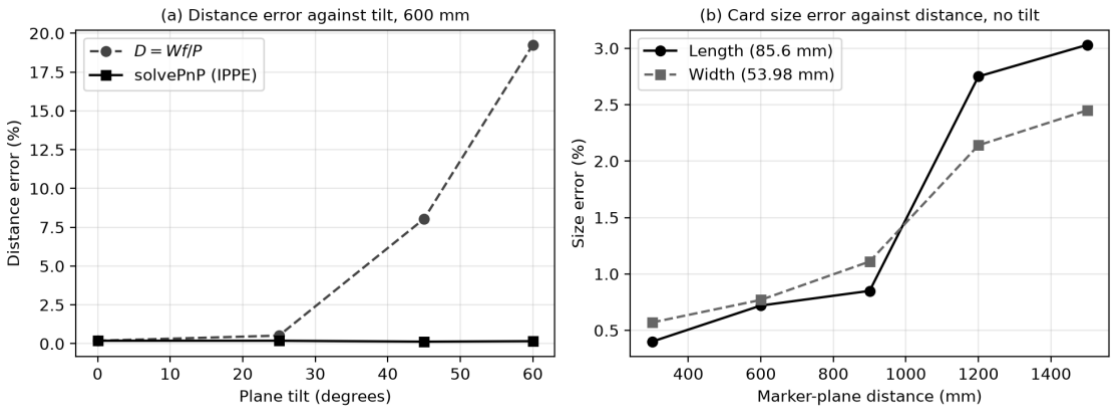


Figure 6.7. Rendered-scene sweeps. (a) Distance error against tilt for similar triangles and solvePnP. (b) Card size error against distance (simulation).

6.4.2 Outline Tracking on DAVIS 2017

`evaluate_tracking.py` clicks the first annotated object of each DAVIS sequence and follows it as in hand mode; every frame is scored against the ground truth (Table 6.6, Figure 6.8).

Table 6.6. Click-and-follow results on DAVIS 2017 (6 sequences × 30 frames)

Outline method	Mean J	On-object	Tracked	Time / frame
Classical (flood fill + edges)	0.05	38 %	0 %	7 ms
Classical, best of 12 settings	0.09	–	3 %	7 ms
MobileSAM (Smart outline)	0.40	62 %	36 %	70 ms

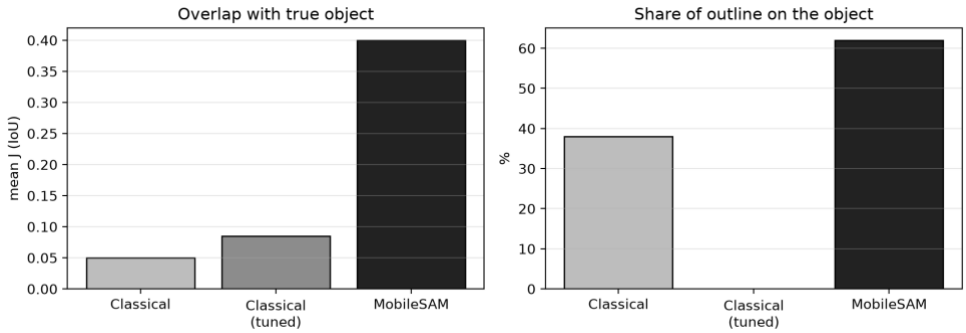


Figure 6.8. Mean J and on-object share for the three outline methods (real video, DAVIS 2017).

MobileSAM followed the bear (J = 0.94) and swan (J = 0.79) well but scored low on bike-packing, bmx and boat, where the object is small, fast or made of several parts (Figure B.5).

6.4.3 Speed

Table 6.7. Processing time per frame (Apple M4 Pro)

Path	Time per frame	Rate
Camera preview (GPU layer)	–	30 FPS
Engine, classical outline	7–16 ms	30 FPS
Engine, scan mode (ORB)	15–20 ms	30 FPS
Engine, MobileSAM outline	70–90 ms	11–14 FPS

6.4.4 Comparison with the Source Paper

Table 6.8 reproduces the accuracy reported by the source paper [1] with a physical 1080p camera. It is given for comparison only and was not reproduced by this project.

Table 6.8. Accuracy reported in the source paper (not a result of this project)

Object	True (mm)	Est. (mm)	Error (mm)	Error (%)
Credit card (width)	85.60	85.82	0.22	+0.25
Credit card (height)	53.98	53.71	0.27	−0.50
Machined steel disk	50.00	50.48	0.48	+0.96
PCB component A	120.50	121.35	0.85	+0.70
Large wooden block	250.00	251.28	1.28	+0.51

6.5 Discussion of Results

The results support three statements. First, the geometry is implemented correctly: on rendered scenes the pipeline recovers distance and tilt to within 1 % up to 60° with PnP, and the size of a flat object to about 1 % within 900 mm, comparable to the 0.25–0.96 % reported by the source paper on real objects. Second, the similar-triangles distance should be used only for a near-frontal marker. Third, a learned outline is clearly better than the classical one at staying on a clicked object in real video, but is not yet reliable on small or multi-part objects.

The limits of the evidence must be stated. No physical-camera measurements have been made; rendered scenes have no sensor noise, glare or auto-exposure; the DAVIS sample is small; MobileSAM was not fine-tuned; timings come from one machine. The hand mode assumes the object and marker are at equal depth, and the no-marker mode inherits the full error of the entered distance. Objects must be flat. The results therefore verify that the method behaves as designed; trials PT-01 to PT-07 are needed to establish real-world accuracy.

Chapter 7: Conclusion and Future Scope

7.1 Conclusion

This project presented Spatial Measure, which measures the real size of a clicked object from the video of one ordinary camera. It corrects lens distortion, uses a printed ArUco marker to define a planar homography to a millimetre scale, outlines the clicked object with a pretrained segmentation model, and fits a rotated rectangle to give length, width and area. Three modes extend the source pipeline to objects larger than the view, to objects in the hand, and to scenes without a marker.

On rendered scenes the IPPE distance and tilt were within 0.7 % and 0.1° up to 60°, and a flat card was measured within about 1.2 % up to 900 mm and 3 % at 1.5 m. The tests also show where the simple method fails: the similar-triangles distance is accurate only for a near-frontal marker, and the no-marker mode inherits the full error of the entered distance. On DAVIS, the pretrained model kept the outline on the clicked object far better than the classical method ($J = 0.40$ against 0.05).

7.2 Future Work

- Carry out the physical trials PT-01 to PT-07: calibration, caliper-referenced accuracy at several distances and tilts, hand, no-marker and scan modes, and lighting.
- Fine-tune the segmentation model on hand-and-object data and evaluate on recorded videos of held objects and on all DAVIS sequences.
- Locate edges to sub-pixel accuracy (Zernike moments or Gaussian edge fitting) to reduce error beyond one metre.
- Show an uncertainty range with each reading from reprojection error, pixel size and distance.
- Port to phones and use ARKit/ARCore or LiDAR for distance, and estimate object height from a second view to remove the flat-object restriction.

References

[1] "Design and implementation of a real-time spatial distance and object dimension estimation system using perspective geometry and planar homography," course reference document supplied with this project (author and publication details are not stated in the document).

[2] D. Eigen, C. Puhrsch, and R. Fergus, "Depth map prediction from a single image using a multi-scale deep network," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 27, 2014, pp. 2366–2374.

[3] R. Ranftl, K. Lasinger, D. Hafner, K. Schindler, and V. Koltun, "Towards robust monocular depth estimation: Mixing datasets for zero-shot cross-dataset transfer," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 44, no. 3, pp. 1623–1637, 2022.

[4] D. C. Brown, "Close-range camera calibration," *Photogramm. Eng.*, vol. 37, no. 8, pp. 855–866, 1971.

[5] R. Y. Tsai, "A versatile camera calibration technique for high-accuracy 3D machine vision metrology using off-the-shelf TV cameras and lenses," *IEEE J. Robot. Autom.*, vol. 3, no. 4, pp. 323–344, 1987.

[6] J. Heikkilä and O. Silvén, "A four-step camera calibration procedure with implicit image correction," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 1997, pp. 1106–1112.

[7] Z. Zhang, "A flexible new technique for camera calibration," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 22, no. 11, pp. 1330–1334, 2000.

[8] A. Criminisi, I. Reid, and A. Zisserman, "Single view metrology," *Int. J. Comput. Vis.*, vol. 40, no. 2, pp. 123–148, 2000.

[9] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed. Cambridge, U.K.: Cambridge Univ. Press, 2004.

[10] H. Kato and M. Billinghurst, "Marker tracking and HMD calibration for a video-based augmented reality conferencing system," in *Proc. 2nd IEEE/ACM Int. Workshop Augmented Reality (IWAR)*, 1999, pp. 85–94.

[11] M. Fiala, "ARTag, a fiducial marker system using digital techniques," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, vol. 2, 2005, pp. 590–596.

[12] E. Olson, "AprilTag: A robust and flexible visual fiducial system," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2011, pp. 3400–3407.

[13] S. Garrido-Jurado, R. Muñoz-Salinas, F. J. Madrid-Cuevas, and M. J. Marín-Jiménez, "Automatic generation and detection of highly reliable fiducial markers under occlusion," *Pattern Recognit.*, vol. 47, no. 6, pp. 2280–2292, 2014.

[14] F. J. Romero-Ramirez, R. Muñoz-Salinas, and R. Medina-Carnicer, "Speeded up detection of squared fiducial markers," *Image Vis. Comput.*, vol. 76, pp. 38–47, 2018.

[15] T. Collins and A. Bartoli, "Infinitesimal plane-based pose estimation," *Int. J. Comput. Vis.*, vol. 109, no. 3, pp. 252–286, 2014.

[16] E. Rublee, V. Rabaud, K. Konolige, and G. Bradski, "ORB: An efficient alternative to SIFT or SURF," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, 2011, pp. 2564–2571.

[17] M. A. Fischler and R. C. Bolles, "Random sample consensus: A paradigm for model fitting with applications to image analysis and automated cartography," *Commun. ACM*, vol. 24, no. 6, pp. 381–395, 1981.

[18] A. Kirillov *et al.*, "Segment anything," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, 2023, pp. 4015–4026.

[19] C. Zhang *et al.*, "Faster segment anything: Towards lightweight SAM for mobile applications," arXiv:2306.14289, 2023.

[20] F. Perazzi, J. Pont-Tuset, B. McWilliams, L. Van Gool, M. Gross, and A. Sorkine-Hornung, "A benchmark dataset and evaluation methodology for video object segmentation," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2016, pp. 724–732.

[21] J. Pont-Tuset, F. Perazzi, S. Caelles, P. Arbeláez, A. Sorkine-Hornung, and L. Van Gool, "The 2017 DAVIS challenge on video object segmentation," arXiv:1704.00675, 2017.

[22] G. Bradski, "The OpenCV library," *Dr. Dobb's J. Softw. Tools*, 2000.

[23] V. Lepetit, F. Moreno-Noguer, and P. Fua, "EPnP: An accurate O(n) solution to the PnP problem," *Int. J. Comput. Vis.*, vol. 81, no. 2, pp. 155–166, 2009.

[24] D. G. Lowe, "Distinctive image features from scale-invariant keypoints," *Int. J. Comput. Vis.*, vol. 60, no. 2, pp. 91–110, 2004.

[25] K. Zuiderveld, "Contrast limited adaptive histogram equalization," in *Graphics Gems IV*, P. S. Heckbert, Ed. San Diego, CA, USA: Academic Press, 1994, pp. 474–485.

[26] J. Canny, "A computational approach to edge detection," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. PAMI-8, no. 6, pp. 679–698, 1986.

[27] N. Otsu, "A threshold selection method from gray-level histograms," *IEEE Trans. Syst., Man, Cybern.*, vol. 9, no. 1, pp. 62–66, 1979.

[28] S. Suzuki and K. Abe, "Topological structural analysis of digitized binary images by border following," *Comput. Vis. Graph. Image Process.*, vol. 30, no. 1, pp. 32–46, 1985.

Appendices

Appendix A: Source Code

The core library and calibration script are listed below. The engine (`engine_server.py`), the OpenCV-window version (`measure.py`), the evaluation scripts, the tests and the Swift application are in the repository (Appendix C).

spatial_measure/estimator.py

```
"""Core pipeline from the paper (Section 3):


    frame -> lens undistortion -> ArUco reference detection -> planar homography
          -> top-down rectification -> edge extraction -> contours
          -> minAreaRect -> pixel-to-mm conversion -> overlay


All measurements assume the objects lie (approximately) on the same plane as the
reference marker. Tall objects violate that assumption (see paper Section 7,
"Non-planar depth variances").
"""
from __future__ import annotations


import math
from dataclasses import dataclass, field


import cv2
import numpy as np



@dataclass
class Measurement:
    length_mm: float  # longer side of the rotated bounding box
    width_mm: float  # shorter side
    angle_deg: float
    center_mm: tuple[float, float]  # on the marker plane, relative to marker top-left corner
    box_image: np.ndarray  # 4x2 box corners in (undistorted) image pixels
    center_image: tuple[float, float]
    outline_image: np.ndarray | None = None  # object outline (simplified polygon), undistorted
image pixels
    area_mm2: float | None = None  # area enclosed by the outline



@dataclass
class FrameResult:
    frame: np.ndarray  # undistorted input frame
    marker_found: bool
    marker_id: int | None = None
    marker_corners: np.ndarray | None = None  # 4x2, image pixels (TL, TR, BR, BL)
    warped: np.ndarray | None = None  # top-down rectified view
    edges: np.ndarray | None = None
    measurements: list[Measurement] = field(default_factory=list)
    distance_similar_mm: float | None = None  # D = W * f / P  (paper Section 5.2)
    distance_pnp_mm: float | None = None  # full pose estimate, robust to tilt
    tilt_deg: float | None = None  # angle between optical axis and marker normal
    warnings: list[str] = field(default_factory=list)
    # Tap-to-measure: the selected object, its seed on the marker plane (mm, for tracking it
    # across frames) and a state: "ok" | "nothing" | "marker" | "lost" | None.
    selected: Measurement | None = None
    seed_mm: tuple[float, float] | None = None
    selection_state: str | None = None
    # Scan mode: how the plane was located ("marker" | "tracked"), the image->canvas homography with
the canvas
    # scale/origin (to map mm back to pixels), and what ScanMap needs to accumulate the object.
    plane_source: str | None = None
    plane_map: tuple | None = None  # (H image->canvas, ppm, origin canvas px of mm (0,0))
    scan_data: dict | None = None
    seed_px: tuple[float, float] | None = None  # follow mode: where to look for the object next
frame (raw pixels)



def valid_small(valid: np.ndarray, w: int, h: int) -> np.ndarray:
    return valid if valid.shape[:2] == (h, w) else cv2.resize(valid, (w, h),
interpolation=cv2.INTER_NEAREST)



class SpatialDimensionEstimator:
    def __init__(
        self,
        marker_length_mm: float = 50.0,
        camera_matrix: np.ndarray | None = None,
        dist_coeffs: np.ndarray | None = None,
        px_per_mm: float = 4.0,
        aruco_dict: int = cv2.aruco.DICT_4X4_50,
        marker_id: int | None = None,
```

```
        min_area_mm2: float = 150.0,
        use_clahe: bool = True,
        canny_thresholds: tuple[int, int] = (50, 150),
        max_canvas_px: int = 2400,
        min_px_per_mm: float = 2.0,
        refine_edges: bool = True,
    ):
        self.marker_length_mm = marker_length_mm
        self.K = camera_matrix
        self.dist = dist_coeffs if dist_coeffs is not None else (np.zeros(5) if camera_matrix is not
None else None)
        self.px_per_mm = px_per_mm
        self.marker_id = marker_id
        self.min_area_mm2 = min_area_mm2
        self.use_clahe = use_clahe
        self.canny_thresholds = canny_thresholds
        self.max_canvas_px = max_canvas_px
        self.min_px_per_mm = min_px_per_mm
        self.refine_edges = refine_edges


        params = cv2.aruco.DetectorParameters()
        params.cornerRefinementMethod = cv2.aruco.CORNER_REFINE_SUBPIX
        self._detector = cv2.aruco.ArucoDetector(cv2.aruco.getPredefinedDictionary(aruco_dict),
params)
        self._clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))
        self.fill_tol = (9, 7, 7)  # flood-fill tolerance in Lab (L, a, b)
        self.hint: np.ndarray | None = None  # last mask from the segmenter: anchors the next frame
on the same object
        self.segmenter = None  # optional learned promptable segmenter (spatial_measure.segmenter),
see _ai_mask


        # Undistortion maps are expensive to build, so cache them per frame size.
        self._maps = None
        self._maps_size = None
        self._new_K = camera_matrix


    # ------------------------------------------------------------------ stages


    def undistort(self, frame: np.ndarray) -> np.ndarray:
        """Removes radial and tangential lens distortion (paper Section 2.2)."""
        if self.K is None:
            return frame
        h, w = frame.shape[:2]
        if self._maps_size != (w, h):
            # alpha=0 keeps only valid pixels at full frame size, so no ROI crop is needed
            # and the principal point stays consistent with new_K.
            self._new_K, _ = cv2.getOptimalNewCameraMatrix(self.K, self.dist, (w, h), 0, (w, h))
            self._maps = cv2.initUndistortRectifyMap(self.K, self.dist, None, self._new_K, (w, h),
cv2.CV_16SC2)
            self._maps_size = (w, h)
        return cv2.remap(frame, self._maps[0], self._maps[1], cv2.INTER_LINEAR)


    def to_raw_pixels(self, pts: np.ndarray) -> np.ndarray:
        """Maps Nx2 points from the undistorted frame back onto the raw (distorted) camera frame,
        e.g. to draw results over a live preview that shows the raw feed."""
        pts = np.asarray(pts, np.float64).reshape(-1, 2)
        if self.K is None or self._maps_size is None:
            return pts
        rays = np.column_stack([pts, np.ones(len(pts))]) @ np.linalg.inv(self._new_K).T
        raw, _ = cv2.projectPoints(rays, np.zeros(3), np.zeros(3), self.K, self.dist)
        return raw.reshape(-1, 2)


    def detect_marker(self, gray: np.ndarray) -> tuple[np.ndarray | None, int | None]:
        corners, ids, _ = self._detector.detectMarkers(gray)
        if ids is None:
            return None, None
        ids = ids.flatten()
        if self.marker_id is not None:
            matches = np.where(ids == self.marker_id)[0]
            if len(matches) == 0:
                return None, None
            i = int(matches[0])
        else:
            i = int(np.argmax([cv2.contourArea(c.reshape(-1, 2)) for c in corners]))
        return corners[i].reshape(4, 2).astype(np.float32), int(ids[i])


    def compute_homography(self, marker_corners: np.ndarray, frame_shape) -> tuple[np.ndarray,
tuple[int, int], np.ndarray, float, list[str]]:
        """Maps the marker plane to a top-down canvas where 1 mm == ppm pixels.
```

```
        The marker's 4 corners give exactly the N >= 4 correspondences the DLT needs
        (paper Section 2.3). The canvas is translated/sized to cover the whole frame; if that
        would exceed max_canvas_px, the resolution is lowered (down to min_px_per_mm) and only
        then is the canvas clipped around the projected image centre.

        Returns (H, canvas size, marker polygon in canvas coords, effective ppm, warnings).
        """
        warnings = []
        h, w = frame_shape[:2]
        frame_pts = np.array([[0, 0, 1], [w - 1, 0, 1], [w - 1, h - 1, 1], [0, h - 1, 1], [w / 2, h
/ 2, 1]], np.float64).T

        ppm = self.px_per_mm
        for _ in range(2):
            s = self.marker_length_mm * ppm
            dst = np.array([[0, 0], [s, 0], [s, s], [0, s]], np.float32)
            H = cv2.getPerspectiveTransform(marker_corners, dst)
            proj = H @ frame_pts
            horizon_visible = np.any(proj[2, :4] <= 1e-9)
            if horizon_visible:
                break
            pts = proj[:2] / proj[2]
            x_min, y_min = pts[:, :4].min(axis=1)
            x_max, y_max = pts[:, :4].max(axis=1)
            extent = max(x_max - x_min, y_max - y_min)
            if extent <= self.max_canvas_px or ppm <= self.min_px_per_mm:
                break
            ppm = max(self.min_px_per_mm, ppm * self.max_canvas_px / extent)

        half = self.max_canvas_px / 2
        if horizon_visible:
            # Part of the image maps to infinity; keep a window around the marker.
            warnings.append("Extreme tilt: horizon visible, canvas clipped around marker")
            x_min, y_min, x_max, y_max = s / 2 - half, s / 2 - half, s / 2 + half, s / 2 + half
        elif extent > self.max_canvas_px:
            cx, cy = pts[:, 4]
            x_min, x_max = max(x_min, cx - half), min(x_max, cx + half)
            y_min, y_max = max(y_min, cy - half), min(y_max, cy + half)

        T = np.array([[1, 0, -x_min], [0, 1, -y_min], [0, 0, 1]], np.float64)
        size = (int(math.ceil(x_max - x_min)), int(math.ceil(y_max - y_min)))
        marker_poly = dst - np.float32([x_min, y_min])
        return T @ H, size, marker_poly, ppm, warnings

    def preprocess(self, warped: np.ndarray) -> tuple[np.ndarray, np.ndarray]:
        """(gray, edge map) of the rectified view.

        CLAHE on the LAB L-channel is the shadow mitigation from paper Section 6.1.
        """
        if self.use_clahe:
            gray = self._clahe.apply(cv2.cvtColor(warped, cv2.COLOR_BGR2LAB)[:, :, 0])
        else:
            gray = cv2.cvtColor(warped, cv2.COLOR_BGR2GRAY)
        blurred = cv2.GaussianBlur(gray, (7, 7), 0)
        edges = cv2.Canny(blurred, *self.canny_thresholds)
        # Morphological closing consolidates broken edges (contour fragmentation).
        kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (5, 5))
        return gray, cv2.morphologyEx(edges, cv2.MORPH_CLOSE, kernel, iterations=2)

    def refine_contour(self, gray: np.ndarray, cnt: np.ndarray) -> np.ndarray:
        """Re-locates the object boundary at the local Otsu threshold.

        Canny + closing places the outer contour 1-2 px outside the true edge (more when the
        rectified view is upsampled), which systematically inflates sizes. Thresholding the
        object's neighbourhood at the midpoint between object and background intensity removes
        that bias. Falls back to the Canny contour when the object is not a clean blob.
        """
        x, y, w, h = cv2.boundingRect(cnt)
        pad = max(10, int(0.15 * max(w, h)))
        x0, y0 = max(0, x - pad), max(0, y - pad)
        x1, y1 = min(gray.shape[1], x + w + pad), min(gray.shape[0], y + h + pad)
        roi = gray[y0:y1, x0:x1]
        _, binary = cv2.threshold(roi, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
        border = np.concatenate([binary[0], binary[-1], binary[:, 0], binary[:, -1]])
        if np.mean(border) > 127:  # background is the bright class -> object is dark
            binary = cv2.bitwise_not(binary)
```

```
        filled = np.zeros_like(binary)
        cv2.drawContours(filled, [cnt - np.int32([x0, y0])], -1, 255, cv2.FILLED)
        candidates, _ = cv2.findContours(cv2.bitwise_and(binary, filled), cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)
        if not candidates:
            return cnt
        best = max(candidates, key=cv2.contourArea)
        ratio = cv2.contourArea(best) / max(cv2.contourArea(cnt), 1.0)
        if not 0.8 <= ratio <= 1.0:
            return cnt
        return best + np.int32([x0, y0])


    @staticmethod
    def _object_contours(edges: np.ndarray, marker_center: np.ndarray, min_area: float) ->
list[np.ndarray]:
        """Outermost contours that are objects rather than the surface the marker sits on.


        A contour enclosing the marker (a sheet of paper, a mat, the table edge) is treated as
        background and we look inside it instead, so objects on a sheet are still found.
        """
        contours, hierarchy = cv2.findContours(edges, cv2.RETR_TREE, cv2.CHAIN_APPROX_SIMPLE)
        if hierarchy is None:
            return []
        hierarchy = hierarchy[0]
        mc = (float(marker_center[0]), float(marker_center[1]))
        found = []
        stack = [i for i in range(len(contours)) if hierarchy[i][3] == -1]
        while stack:
            i = stack.pop()
            if cv2.pointPolygonTest(contours[i], mc, False) >= 0:
                child = hierarchy[i][2]
                while child != -1:
                    stack.append(child)
                    child = hierarchy[child][0]
            elif cv2.contourArea(contours[i]) >= min_area:
                found.append(contours[i])
        return found


    def measure_contours(self, gray: np.ndarray, edges: np.ndarray, H: np.ndarray, marker_poly:
np.ndarray, ppm: float) -> list[Measurement]:
        H_inv = np.linalg.inv(H)
        origin = marker_poly[0]
        # Grow the marker square 20% so the marker itself (and its paper border) is skipped.
        mc = marker_poly.mean(axis=0)
        marker_zone = ((marker_poly - mc) * 1.2 + mc).astype(np.float32)


        results = []
        for cnt in self._object_contours(edges, mc, self.min_area_mm2 * ppm * ppm):
            if self.refine_edges:
                cnt = self.refine_contour(gray, cnt)
            rect = cv2.minAreaRect(cnt)
            (cx, cy), (w_px, h_px), angle = rect
            if cv2.pointPolygonTest(marker_zone, (float(cx), float(cy)), False) >= 0:
                continue
            if min(w_px, h_px) < 2 * ppm:  # slivers: shadows, table seams, stray edge fragments
                continue
            box = cv2.boxPoints(rect).astype(np.float32)
            box_img = cv2.perspectiveTransform(box.reshape(-1, 1, 2), H_inv).reshape(-1, 2)
            c_img = cv2.perspectiveTransform(np.array([[[cx, cy]]], np.float32), H_inv).reshape(2)
            results.append(
                Measurement(
                    length_mm=max(w_px, h_px) / ppm,
                    width_mm=min(w_px, h_px) / ppm,
                    angle_deg=angle,
                    center_mm=((cx - origin[0]) / ppm, (cy - origin[1]) / ppm),
                    box_image=box_img,
                    center_image=(float(c_img[0]), float(c_img[1])),
                )
            )
        results.sort(key=lambda m: m.length_mm * m.width_mm, reverse=True)
        return results


    def estimate_distance(self, marker_corners: np.ndarray) -> tuple[float | None, float | None,
float | None]:
        """Camera-to-marker distance.


        Returns (similar-triangles D = W*f/P, solvePnP distance, tilt in degrees).
        Similar triangles is only valid when the marker faces the camera; PnP is not.
```

```
        """
        if self._new_K is None:
            return None, None, None
        K = self._new_K
        f = (K[0, 0] + K[1, 1]) / 2
        sides = np.linalg.norm(marker_corners - np.roll(marker_corners, -1, axis=0), axis=1)
        d_similar = self.marker_length_mm * f / sides.mean()


        L = self.marker_length_mm / 2
        obj = np.array([[-L, L, 0], [L, L, 0], [L, -L, 0], [-L, -L, 0]], np.float32)
        ok, rvec, tvec = cv2.solvePnP(obj, marker_corners, K, np.zeros(5),
flags=cv2.SOLVEPNP_IPPE_SQUARE)
        if not ok:
            return d_similar, None, None
        R, _ = cv2.Rodrigues(rvec)
        tilt = math.degrees(math.acos(min(1.0, abs(R[2, 2]))))
        return d_similar, float(np.linalg.norm(tvec)), tilt



    # ------------------------------------------------------------ tap to measure


    def _region_at(self, warped: np.ndarray, edges: np.ndarray, valid: np.ndarray, seed:
tuple[float, float], max_frac: float = 0.5, skin: bool = False) -> np.ndarray | None:
        """Region of similar colour around the seed, walled in by the Canny edges.


        Works on a downscaled copy (cheap enough to run every frame) and returns the outline in
        canvas pixels, or None when the fill leaks across the scene or is too small to be an object.
        """
        h, w = warped.shape[:2]
        k = min(1.0, 700.0 / max(h, w))
        small = cv2.resize(warped, None, fx=k, fy=k, interpolation=cv2.INTER_AREA) if k < 1 else
warped
        sh, sw = small.shape[:2]
        lab = cv2.GaussianBlur(cv2.cvtColor(small, cv2.COLOR_BGR2LAB), (5, 5), 0)
        wall = cv2.resize(edges, (sw, sh), interpolation=cv2.INTER_NEAREST) if k < 1 else
edges.copy()
        wall = cv2.dilate(wall, np.ones((3, 3), np.uint8))
        wall = np.where(wall > 0, 1, 0).astype(np.uint8)
        wall[valid_small(valid, sw, sh) == 0] = 1
        if skin:
            # Fingers holding the object are walled off so the fill does not run into the hand.
            ycc = cv2.cvtColor(small, cv2.COLOR_BGR2YCrCb)
            hand = cv2.inRange(ycc, (0, 135, 85), (255, 175, 125))
            hand = cv2.morphologyEx(hand, cv2.MORPH_OPEN, np.ones((3, 3), np.uint8))
            hand[max(0, int(seed[1] * k) - 1):int(seed[1] * k) + 2, max(0, int(seed[0] * k) -
1):int(seed[0] * k) + 2] = 0
            wall[hand > 0] = 1
        mask = np.zeros((sh + 2, sw + 2), np.uint8)
        mask[1:-1, 1:-1] = wall
        sx, sy = int(round(seed[0] * k)), int(round(seed[1] * k))
        if not (0 <= sx < sw and 0 <= sy < sh) or wall[sy, sx]:
            # seed sits exactly on an edge pixel: nudge to a free neighbour
            ok = False
            for dx, dy in ((1, 0), (-1, 0), (0, 1), (0, -1), (2, 2), (-2, -2), (2, -2), (-2, 2)):
                x2, y2 = sx + dx, sy + dy
                if 0 <= x2 < sw and 0 <= y2 < sh and not wall[y2, x2]:
                    sx, sy, ok = x2, y2, True
                    break
            if not ok:
                return None
        flags = 4 | cv2.FLOODFILL_MASK_ONLY | (255 << 8)
        cv2.floodFill(lab, mask, (sx, sy), 0, self.fill_tol, self.fill_tol, flags)
        region = (mask[1:-1, 1:-1] == 255).astype(np.uint8) * 255
        region = cv2.morphologyEx(region, cv2.MORPH_CLOSE,
cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (7, 7)))
        # The walls keep the fill ~1.5 px short of the true edge; grow back, refine_contour trims
the excess.
        region = cv2.dilate(region, cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5)))
        cnts, _ = cv2.findContours(region, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
        cnts = [c for c in cnts if cv2.pointPolygonTest(c, (float(sx), float(sy)), False) >= 0]
        if not cnts:
            return None
        cnt = max(cnts, key=cv2.contourArea)
        frac = cv2.contourArea(cnt) / float(max(1, np.count_nonzero(valid_small(valid, sw, sh))))
        if frac > max_frac:  # leaked into the background
            return None
        return (cnt.reshape(-1, 1, 2).astype(np.float32) / k).astype(np.int32)
```

```
    def _contour_at(self, edges: np.ndarray, seed: tuple[float, float], min_area: float) ->
np.ndarray | None:
        """Fallback: the smallest edge contour that contains the seed and is big enough."""
        contours, _ = cv2.findContours(edges, cv2.RETR_TREE, cv2.CHAIN_APPROX_SIMPLE)
        best, best_a = None, None
        for c in contours:
            a = cv2.contourArea(c)
            if a >= min_area and cv2.pointPolygonTest(c, seed, False) >= 0 and (best_a is None or a
< best_a):
                best, best_a = c, a
        return best



    def _ai_mask(self, frame: np.ndarray, seed: tuple[float, float], skin: bool) -> np.ndarray |
None:
        """Mask (frame pixels) of the object under `seed` from the learned segmenter, with the hand
taken out."""
        m = self.segmenter.segment(frame, seed, self._hint_points(frame.shape[:2]))
        if m is None:
            self.hint = None
            return None
        m = m.astype(np.uint8)
        if skin:
            hand = cv2.inRange(cv2.cvtColor(frame, cv2.COLOR_BGR2YCrCb), (0, 135, 85), (255, 175,
125))
            hand = cv2.morphologyEx(hand, cv2.MORPH_OPEN, np.ones((5, 5), np.uint8))
            m[hand > 0] = 0
        m = cv2.morphologyEx(m, cv2.MORPH_OPEN, np.ones((5, 5), np.uint8))
        n, lab, stats, _ = cv2.connectedComponentsWithStats(m, connectivity=8)
        if n < 2:
            self.hint = None
            return None
        sx, sy = int(round(seed[0])), int(round(seed[1]))
        inside = 0 <= sx < m.shape[1] and 0 <= sy < m.shape[0] and lab[sy, sx] > 0
        idx = int(lab[sy, sx]) if inside else 1 + int(np.argmax(stats[1:, cv2.CC_STAT_AREA]))
        self.hint = (lab == idx).astype(np.uint8)
        return self.hint


    def _hint_points(self, shape, n: int = 4):
        """A few well-spread points deep inside the previous mask (farthest-point sampling), as
extra clicks."""
        if self.hint is None or self.hint.shape != tuple(shape):
            return []
        d = cv2.distanceTransform(self.hint, cv2.DIST_L2, 3)
        if d.max() < 4:
            return []
        ys, xs = np.nonzero(d >= 0.5 * d.max())
        if len(xs) == 0:
            return []
        pts = np.column_stack([xs, ys]).astype(np.float32)
        if len(pts) > 4000:
            pts = pts[np.linspace(0, len(pts) - 1, 4000).astype(int)]
        chosen = [pts[int(np.argmax(d[pts[:, 1].astype(int), pts[:, 0].astype(int)]))]]
        dist = np.linalg.norm(pts - chosen[0], axis=1)
        for _ in range(n - 1):
            chosen.append(pts[int(np.argmax(dist))])
            dist = np.minimum(dist, np.linalg.norm(pts - chosen[-1], axis=1))
        return [(float(x), float(y)) for x, y in chosen[1:]]


    def _largest_contour(self, mask: np.ndarray, seed=None) -> np.ndarray | None:
        cnts, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
        if not cnts:
            return None
        if seed is not None:
            hit = [c for c in cnts if cv2.pointPolygonTest(c, (float(seed[0]), float(seed[1])),
False) >= 0]
            cnts = hit or cnts
        return max(cnts, key=cv2.contourArea)


    def segment_image(self, frame: np.ndarray, seed: tuple[float, float], ppm: float = 1.0, skin:
bool = True):
        """Outline of the object under `seed` in a plain image (no plane, no marker). Returns
(contour, gray) with
        contour None when nothing sensible is found. Uses the learned segmenter when one is
attached."""
        h, w = frame.shape[:2]
        gray, edges = self.preprocess(frame)
        cnt = None
        if self.segmenter is not None:
            m = self._ai_mask(frame, seed, skin)
```

```
            cnt = None if m is None else self._largest_contour(m, seed)
            if cnt is not None and cv2.contourArea(cnt) < 20.0 * ppm * ppm:
                cnt = None
        if cnt is None:
            valid = np.full((h, w), 255, np.uint8)
            valid[:3], valid[-3:], valid[:, :3], valid[:, -3:] = 0, 0, 0, 0
            min_area = 20.0 * ppm * ppm
            cnt = self._region_at(frame, edges, valid, seed, 0.97, skin)
            if cnt is None or cv2.contourArea(cnt) < min_area:
                cnt = self._contour_at(edges, seed, min_area)
            if cnt is not None and cv2.contourArea(cnt) < 150.0 * ppm * ppm:
                outer = self._contour_at(edges, seed, 3.0 * cv2.contourArea(cnt))
                if outer is not None and cv2.contourArea(outer) <= 60.0 * cv2.contourArea(cnt):
                    cnt = outer
            if cnt is not None and self.refine_edges:
                cnt = self.refine_contour(gray, cnt)
        return cnt, gray


    def select_object(self, warped, gray, edges, valid, H, marker_poly, ppm, seed, max_frac=0.5,
skin=False, ai=None):
        """Outlines and measures the object under `seed` (canvas pixels). Returns Measurement or
None."""
        mc = marker_poly.mean(axis=0)
        marker_zone = ((marker_poly - mc) * 1.2 + mc).astype(np.float32)
        if cv2.pointPolygonTest(marker_zone, (float(seed[0]), float(seed[1])), False) >= 0:
            return "marker"
        min_area = 20.0 * ppm * ppm  # 20 mm2
        cnt = ai(seed) if ai is not None else None  # learned segmentation first, classical as the
fallback
        if cnt is not None and cv2.contourArea(cnt) < min_area:
            cnt = None
        ai_used = cnt is not None
        if cnt is None:
            cnt = self._region_at(warped, edges, valid, seed, max_frac, skin)
        if not ai_used and (cnt is None or cv2.contourArea(cnt) < min_area):
            cnt = self._contour_at(edges, seed, min_area)
        if not ai_used and cnt is not None and cv2.contourArea(cnt) < 150.0 * ppm * ppm:
            # A small patch is probably print or a mark on something bigger (a letter on a card):
            # prefer the enclosing outline when it is of comparable scale.
            outer = self._contour_at(edges, seed, 3.0 * cv2.contourArea(cnt))
            if outer is not None and cv2.contourArea(outer) <= 60.0 * cv2.contourArea(cnt):
                cnt = outer
        if cnt is None:
            return None
        if self.refine_edges and not ai_used:
            cnt = self.refine_contour(gray, cnt)
        rect = cv2.minAreaRect(cnt)
        (cx, cy), (w_px, h_px), angle = rect
        if min(w_px, h_px) < 2 * ppm:
            return None
        H_inv = np.linalg.inv(H)
        origin = marker_poly[0]
        box = cv2.boxPoints(rect).astype(np.float32)
        box_img = cv2.perspectiveTransform(box.reshape(-1, 1, 2), H_inv).reshape(-1, 2)
        c_img = cv2.perspectiveTransform(np.array([[[cx, cy]]], np.float32), H_inv).reshape(2)
        poly = cv2.approxPolyDP(cnt, 0.008 * cv2.arcLength(cnt, True), True).astype(np.float32)
        outline = cv2.perspectiveTransform(poly.reshape(-1, 1, 2), H_inv).reshape(-1, 2)
        return Measurement(
            length_mm=max(w_px, h_px) / ppm, width_mm=min(w_px, h_px) / ppm, angle_deg=angle,
            center_mm=((cx - origin[0]) / ppm, (cy - origin[1]) / ppm), box_image=box_img,
            center_image=(float(c_img[0]), float(c_img[1])), outline_image=outline,
            area_mm2=float(cv2.contourArea(cnt)) / (ppm * ppm),
        ), cnt


    def _inner_point(self, cnt: np.ndarray) -> tuple[float, float]:
        """A point well inside the outline (peak of the distance transform), used to follow the
object."""
        x, y, w, h = cv2.boundingRect(cnt)
        m = np.zeros((h + 2, w + 2), np.uint8)
        cv2.drawContours(m, [cnt - np.int32([x - 1, y - 1])], -1, 255, cv2.FILLED)
        d = cv2.distanceTransform(m, cv2.DIST_L2, 3)
        _, _, _, loc = cv2.minMaxLoc(d)
        return float(loc[0] + x - 1), float(loc[1] + y - 1)


    def process_nomarker(self, frame: np.ndarray, seed_px: tuple[float, float], distance_mm: float,
                         hfov_deg: float = 65.0) -> FrameResult:
        """No marker: scale comes from an assumed camera-to-object distance and the lens' focal
length
        (from the calibration if there is one, else from the horizontal field of view). The object
is taken to
```

```
        face the camera, so this is an estimate: size error is about the distance error, in
proportion."""
        frame = self.undistort(frame)
        h, w = frame.shape[:2]
        f = float(self._new_K[0, 0]) if self._new_K is not None else (w / 2) /
math.tan(math.radians(hfov_deg) / 2)
        ppm = f / max(distance_mm, 1.0)  # image pixels per mm at the object's distance
        p = np.array([[seed_px]], np.float32)
        if self.K is not None:
            p = cv2.undistortPoints(p, self.K, self.dist, P=self._new_K)
        seed = (float(p.reshape(2)[0]), float(p.reshape(2)[1]))
        warnings = [f"Estimate: assumes the object is {distance_mm / 10:.0f} cm from the camera"]
        cnt, _ = self.segment_image(frame, seed, ppm, skin=True)
        if cnt is None:
            return FrameResult(frame=frame, marker_found=False, selection_state="nothing",
warnings=warnings)
        rect = cv2.minAreaRect(cnt)
        (cx, cy), (w_px, h_px), angle = rect
        if min(w_px, h_px) < 2 * ppm:
            return FrameResult(frame=frame, marker_found=False, selection_state="nothing",
warnings=warnings)
        poly = cv2.approxPolyDP(cnt, 0.008 * cv2.arcLength(cnt, True), True).reshape(-1,
2).astype(np.float32)
        m = Measurement(
            length_mm=max(w_px, h_px) / ppm, width_mm=min(w_px, h_px) / ppm, angle_deg=angle,
            center_mm=(cx / ppm, cy / ppm), box_image=cv2.boxPoints(rect).astype(np.float32),
            center_image=(float(cx), float(cy)), outline_image=poly,
area_mm2=float(cv2.contourArea(cnt)) / (ppm * ppm),
        )
        ix, iy = self._inner_point(cnt)
        nxt = tuple(float(v) for v in self.to_raw_pixels(np.array([[ix, iy]]))[0])
        return FrameResult(frame=frame, marker_found=False, selected=m, selection_state="ok",
                           seed_px=nxt, plane_source="assumed", warnings=warnings)

    # ------------------------------------------------------------------ pipeline


    def process(self, frame: np.ndarray, seed_px: tuple[float, float] | None = None,
                seed_mm: tuple[float, float] | None = None, tracker=None,
                extra_seeds_mm=None, validate=None, follow: bool = False, skin: bool = False) ->
FrameResult:
        """seed_px: a tap in raw camera pixels. seed_mm: a previously selected object, as a point on
the
        marker plane. With either, only that object is outlined and measured."""
        frame = self.undistort(frame)
        gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
        corners, mid = self.detect_marker(gray)
        source = "marker"
        if tracker is not None:
            # Scan mode: keep the plane located even when the marker is out of view.
            if corners is not None:
                tracker.anchor(gray, corners)
            else:
                M = tracker.track(gray)
                if M is not None:
                    corners, mid, source = tracker.virtual_corners(M), None, "tracked"
        if corners is None:
            return FrameResult(frame=frame, marker_found=False, warnings=["Reference marker not
found"])


        H, size, marker_poly, ppm, warnings = self.compute_homography(corners, frame.shape)
        warped = cv2.warpPerspective(frame, H, size)
        # Ignore the artificial edge between real pixels and the black fill outside the frame.
        valid = cv2.warpPerspective(np.full(frame.shape[:2], 255, np.uint8), H, size)
        valid = cv2.erode(valid, np.ones((9, 9), np.uint8))
        gray_w, edges = self.preprocess(warped)
        edges = cv2.bitwise_and(edges, valid)


        d_similar, d_pnp, tilt = self.estimate_distance(corners) if source == "marker" else (None,
None, None)
        if tilt is not None and tilt > 65:
            warnings.append(f"Pitch {tilt:.0f} deg > 65 deg: perspective compression, accuracy
degraded")


        selected, new_seed, state, scan_data, next_px = None, None, None, None, None
        if seed_px is not None or seed_mm is not None:
            origin = marker_poly[0]
            if seed_px is not None:
                p = np.array([[seed_px]], np.float32)
                if self.K is not None:
                    p = cv2.undistortPoints(p, self.K, self.dist, P=self._new_K)
```

```
                seed_c = cv2.perspectiveTransform(p.reshape(1, 1, 2), H).reshape(2)
            else:
                seed_c = np.array([seed_mm[0] * ppm + origin[0], seed_mm[1] * ppm + origin[1]],
np.float32)
            # The tracked seed first; in scan mode, when it has left the view, other known points of
the same
            # object that are in view (extra_seeds_mm) take over.
            ai = None
            if self.segmenter is not None:
                H_inv_frame = np.linalg.inv(H)

                def ai(sc, _frame=frame, _H=H, _Hi=H_inv_frame, _size=size, _valid=valid):
                    si = cv2.perspectiveTransform(np.array([[sc]], np.float32), _Hi).reshape(2)
                    m = self._ai_mask(_frame, (float(si[0]), float(si[1])), skin)
                    if m is None:
                        return None
                    mc = cv2.warpPerspective(m, _H, _size, flags=cv2.INTER_NEAREST) & (_valid >
0).astype(np.uint8)
                    return self._largest_contour(mc, sc)

            tries = [seed_c]
            for sm in (extra_seeds_mm or []):
                tries.append(np.array([sm[0] * ppm + origin[0], sm[1] * ppm + origin[1]],
np.float32))
            res, used = None, None
            for k, sc in enumerate(tries):
                x, y = int(sc[0]), int(sc[1])
                if not (8 <= x < size[0] - 8 and 8 <= y < size[1] - 8) or valid[y, x] == 0:
                    continue
                res = self.select_object(warped, gray_w, edges, valid, H, marker_poly, ppm,
                                         (float(sc[0]), float(sc[1])), 0.97 if tracker is not None
else 0.5, skin, ai)
                if res is not None and res != "marker":
                    if validate is not None and not validate(res[1], valid, ppm, marker_poly[0]):
                        res = None
                        continue
                    used = sc
                    break
                if k > 0:
                    res = None
            if res == "marker":
                state = "marker"
            elif res is None:
                state = "nothing" if seed_px is not None else "lost"
                if seed_mm is not None:
                    new_seed = seed_mm
            else:
                selected, cnt = res
                if tracker is not None:
                    scan_data = {"cnt": cnt, "valid": valid, "ppm": ppm, "origin": marker_poly[0]}
                if not follow and cv2.pointPolygonTest(cnt, (float(used[0]), float(used[1])), False)
>= 0:
                    ix, iy = float(used[0]), float(used[1])  # keep a seed that is still inside:
steadier than re-picking
                else:
                    ix, iy = self._inner_point(cnt)
                new_seed = ((ix - origin[0]) / ppm, (iy - origin[1]) / ppm)
                state = "ok"
                if follow:
                    p_img = cv2.perspectiveTransform(np.array([[[ix, iy]]], np.float32),
np.linalg.inv(H)).reshape(1, 2)
                    next_px = tuple(float(v) for v in self.to_raw_pixels(p_img)[0])

        return FrameResult(
            frame=frame,
            marker_found=source == "marker",
            marker_id=mid,
            marker_corners=corners,
            warped=warped,
            edges=edges,
            measurements=[] if (seed_px is not None or seed_mm is not None) else
self.measure_contours(gray_w, edges, H, marker_poly, ppm),
            selected=selected, seed_mm=new_seed, selection_state=state,
            plane_source=source, plane_map=(H, ppm, marker_poly[0]), scan_data=scan_data,
seed_px=next_px,
            distance_similar_mm=d_similar,
            distance_pnp_mm=d_pnp,
            tilt_deg=tilt,
            warnings=warnings,
        )
```

```
def draw_overlay(result: FrameResult, show_pairwise: bool = True) -> np.ndarray:
    out = result.frame.copy()
    if result.marker_found:
        cv2.polylines(out, [result.marker_corners.astype(np.int32)], True, (255, 160, 0), 2)
        tl = result.marker_corners[0].astype(int)
        cv2.putText(out, f"REF id={result.marker_id}", (tl[0], tl[1] - 8), cv2.FONT_HERSHEY_SIMPLEX,
0.5, (255, 160, 0), 2)

    for i, m in enumerate(result.measurements):
        cv2.polylines(out, [m.box_image.astype(np.int32)], True, (0, 255, 0), 2)
        cx, cy = int(m.center_image[0]), int(m.center_image[1])
        label = f"#{i + 1} {m.length_mm:.1f} x {m.width_mm:.1f} mm"
        _put_label(out, label, (cx - 70, cy))

    if show_pairwise and len(result.measurements) >= 2:
        ms = sorted(result.measurements, key=lambda m: m.center_image[0])
        for a, b in zip(ms, ms[1:]):
            d = math.dist(a.center_mm, b.center_mm)
            pa = tuple(int(v) for v in a.center_image)
            pb = tuple(int(v) for v in b.center_image)
            cv2.line(out, pa, pb, (0, 200, 255), 1, cv2.LINE_AA)
            mid = ((pa[0] + pb[0]) // 2, (pa[1] + pb[1]) // 2 - 6)
            _put_label(out, f"{d:.0f} mm", mid, color=(0, 200, 255))

    y = 24
    lines = []
    if result.distance_pnp_mm is not None:
        lines.append(f"Marker distance: {result.distance_pnp_mm:.0f} mm (PnP) |
{result.distance_similar_mm:.0f} mm (W*f/P)")
    if result.tilt_deg is not None:
        lines.append(f"Plane tilt: {result.tilt_deg:.0f} deg")
    lines += result.warnings
    for line in lines:
        _put_label(out, line, (10, y), color=(0, 0, 255) if line in result.warnings else (255, 255,
255))
        y += 22
    return out



def _put_label(img, text, org, color=(255, 255, 255)):
    cv2.putText(img, text, org, cv2.FONT_HERSHEY_SIMPLEX, 0.55, (0, 0, 0), 4, cv2.LINE_AA)
    cv2.putText(img, text, org, cv2.FONT_HERSHEY_SIMPLEX, 0.55, color, 1, cv2.LINE_AA)
```

spatial_measure/segmenter.py

```
"""Learned promptable segmentation: click a point, get the mask of the object under it.


MobileSAM (a distilled Segment Anything model, trained on the SA-1B dataset: 11 M images, 1.1 B
masks) is run
through ultralytics. Its job here is only to say *which pixels are the object*; the geometry
(homography, scale,
minAreaRect) stays exactly as before, so the measurement is still metric.
"""
from __future__ import annotations


import os


import cv2
import numpy as np


WEIGHTS = os.path.join(os.path.dirname(os.path.dirname(os.path.abspath(__file__))), "models",
"mobile_sam.pt")



class SamSegmenter:
    def __init__(self, weights: str = WEIGHTS):
        import torch
        from ultralytics import SAM


        os.makedirs(os.path.dirname(weights), exist_ok=True)
        self.device = "mps" if torch.backends.mps.is_available() else "cpu"
        self.model = SAM(weights)  # downloads the official MobileSAM weights on first use


    def segment(self, frame_bgr: np.ndarray, point: tuple[float, float], extra_points=()) ->
np.ndarray | None:
        """Mask of the object at `point`. extra_points: more positive clicks on the *same* object
(taken from the
        previous frame's mask), which keeps the answer on the whole object instead of drifting to a
part of it."""
        pts = [[float(point[0]), float(point[1])]] + [[float(x), float(y)] for x, y in extra_points]
        res = self.model.predict(frame_bgr, points=[pts], labels=[[1] * len(pts)],
device=self.device, verbose=False)
        if not res or res[0].masks is None or len(res[0].masks.data) == 0:
            return None
        m = res[0].masks.data[0].cpu().numpy()
        if m.shape != frame_bgr.shape[:2]:
            m = cv2.resize(m, (frame_bgr.shape[1], frame_bgr.shape[0]),
interpolation=cv2.INTER_NEAREST)
        return (m > 0.5).astype(np.uint8)
```

spatial_measure/scan.py

```
"""Measuring objects that are bigger than one camera frame.


Everything is expressed in millimetres on the marker plane (origin = marker top-left corner), which
does not move when the camera does. Two pieces make that work:


* PlaneTracker keeps knowing where the plane is while the marker is out of view. Whenever the marker
  is visible it gives an exact image->mm homography; when it is not, ORB features on the table are
matched
  against stored keyframes whose homography is known, and the two homographies are chained.
* ScanMap accumulates, frame after frame, which parts of the plane have been *seen* and which of
those
  belong to the tapped object, then measures the object from the consensus. It also knows whether
the
  object's outline touches an unseen area, i.e. whether the size so far is only a lower bound.
"""
from __future__ import annotations


import base64


import cv2
import numpy as np



class PlaneTracker:
    def __init__(self, marker_mm: float, max_keyframes: int = 40, feature_width: int = 960):
        self.marker_mm = marker_mm
        self.max_keyframes = max_keyframes
        self.feature_width = feature_width
        self._orb = cv2.ORB_create(1200, scaleFactor=1.2, nlevels=6, fastThreshold=12)
        self._bf = cv2.BFMatcher(cv2.NORM_HAMMING)
        self.reset()


    def reset(self):
        self.keyframes: list[dict] = []
        self.M: np.ndarray | None = None  # image (undistorted px) -> plane mm, for the latest frame
        self.ready = False
        self.frames_since_marker = 0
        self.last_center: np.ndarray | None = None


    # ------------------------------------------------------------------ features


    def _features(self, gray: np.ndarray):
        s = min(1.0, self.feature_width / gray.shape[1])
        small = cv2.resize(gray, None, fx=s, fy=s, interpolation=cv2.INTER_AREA) if s < 1 else gray
        kps, des = self._orb.detectAndCompute(small, None)
        if des is None or len(kps) < 12:
            return None, None
        return (np.float32([k.pt for k in kps]) / s), des


    @staticmethod
    def _mm(M: np.ndarray, pts) -> np.ndarray:
        return cv2.perspectiveTransform(np.float32(pts).reshape(-1, 1, 2), M).reshape(-1, 2)


    def _maybe_keyframe(self, gray: np.ndarray, M: np.ndarray):
        h, w = gray.shape[:2]
        c = self._mm(M, [(w / 2, h / 2)])[0]
        a, b = self._mm(M, [(0, h / 2), (w, h / 2)])
        span = float(np.linalg.norm(a - b))
        if not np.isfinite(span) or span <= 0:
            return
        if any(np.linalg.norm(k["center"] - c) < 0.25 * span for k in self.keyframes):
            return
        pts, des = self._features(gray)
        if pts is None:
            return
        self.keyframes.append({"pts": pts, "des": des, "M": M.copy(), "center": c})
        if len(self.keyframes) > self.max_keyframes:
            self.keyframes.pop(1 if len(self.keyframes) > 2 else 0)  # keep the first (marker)
keyframe


    # ------------------------------------------------------------------ public


    def anchor(self, gray: np.ndarray, corners: np.ndarray) -> np.ndarray:
```

```
        """The marker is visible: exact plane homography."""
        L = self.marker_mm
        M = cv2.getPerspectiveTransform(np.float32(corners), np.float32([[0, 0], [L, 0], [L, L], [0,
L]]))
        self.M, self.ready, self.frames_since_marker = M, True, 0
        h, w = gray.shape[:2]
        self.last_center = self._mm(M, [(w / 2, h / 2)])[0]
        self._maybe_keyframe(gray, M)
        return M

    def track(self, gray: np.ndarray) -> np.ndarray | None:
        """The marker is out of view: chain a homography from the best-matching keyframe."""
        if not self.ready or not self.keyframes:
            return None
        pts, des = self._features(gray)
        if pts is None:
            return None
        order = sorted(self.keyframes, key=lambda k: np.linalg.norm(k["center"] - self.last_center)
                       if self.last_center is not None else 0.0)
        for kf in order[:4]:
            pairs = self._bf.knnMatch(des, kf["des"], k=2)
            good = [m for m, n in (p for p in pairs if len(p) == 2) if m.distance < 0.75 *
n.distance]
            if len(good) < 20:
                continue
            src = pts[[m.queryIdx for m in good]]
            dst = kf["pts"][[m.trainIdx for m in good]]
            Hk, inl = cv2.findHomography(src, dst, cv2.RANSAC, 3.0)
            if Hk is None or int(inl.sum()) < 18 or inl.mean() < 0.35:
                continue
            scale = float(np.sqrt(abs(np.linalg.det(Hk[:2, :2]))))
            if not 0.4 < scale < 2.5:
                continue
            M = kf["M"] @ Hk
            h, w = gray.shape[:2]
            c = self._mm(M, [(w / 2, h / 2)])[0]
            if not np.all(np.isfinite(c)):
                continue
            self.M, self.last_center = M, c
            self.frames_since_marker += 1
            self._maybe_keyframe(gray, M)
            return M
        return None

    def virtual_corners(self, M: np.ndarray) -> np.ndarray:
        """Where the marker's corners would be in the current image (possibly outside it)."""
        L = self.marker_mm
        return self._mm(np.linalg.inv(M), [(0, 0), (L, 0), (L, L), (0, L)]).astype(np.float32)



class ScanMap:
    def __init__(self, ppm: float = 2.0, half_mm: float = 1000.0, freeze: int = 60):
        self.ppm, self.freeze = ppm, freeze
        self.G = int(2 * half_mm * ppm)
        self.c = half_mm * ppm
        self.reset()

    def reset(self):
        self.obj = np.zeros((self.G, self.G), np.uint8)
        self.obs = np.zeros((self.G, self.G), np.uint8)
        self.bbox: tuple[int, int, int, int] | None = None  # x0, y0, x1, y1 of the object votes
        self._cons = None  # latest consensus object (x0, y0, x1, y1, mask), for consistency checks


    def _rasterize(self, cnt, valid, canvas_ppm, origin):
        """Frame's object mask and observed mask, warped into the global map. Returns (x0, y0, x1,
y1, obs, obj)."""
        k = self.ppm / canvas_ppm
        tx, ty = self.c - origin[0] * k, self.c - origin[1] * k
        ch, cw = valid.shape[:2]
        x0, y0 = max(0, int(np.floor(tx))), max(0, int(np.floor(ty)))
        x1, y1 = min(self.G, int(np.ceil(k * cw + tx))), min(self.G, int(np.ceil(k * ch + ty)))
        if x1 - x0 < 2 or y1 - y0 < 2:
            return None
        A = np.float64([[k, 0, tx - x0], [0, k, ty - y0]])
        size = (x1 - x0, y1 - y0)
        obs_m = cv2.warpAffine((valid > 0).astype(np.uint8), A, size, flags=cv2.INTER_NEAREST)
        full = np.zeros((ch, cw), np.uint8)
        cv2.drawContours(full, [cnt], -1, 1, cv2.FILLED)
        obj_m = cv2.warpAffine(full, A, size, flags=cv2.INTER_NEAREST) & obs_m
```

```
        return x0, y0, x1, y1, obs_m, obj_m


    def consistent(self, cnt, valid, canvas_ppm, origin) -> bool:
        """Does this frame's object outline agree with what earlier frames established for the parts
it sees again?
        Rejects a frame that latched onto a mark or letter instead of the object."""
        if self._cons is None:
            return True
        r = self._rasterize(cnt, valid, canvas_ppm, origin)
        if r is None:
            return True
        x0, y0, x1, y1, obs_m, obj_m = r
        cx0, cy0, cx1, cy1, comp = self._cons
        ix0, iy0, ix1, iy1 = max(x0, cx0), max(y0, cy0), min(x1, cx1), min(y1, cy1)
        if ix1 <= ix0 or iy1 <= iy0:
            return True
        known = comp[iy0 - cy0:iy1 - cy0, ix0 - cx0:ix1 - cx0].astype(bool) & obs_m[iy0 - y0:iy1 -
y0, ix0 - x0:ix1 - x0].astype(bool)
        n = int(known.sum())
        if n < 200:
            return True
        hit = int((known & obj_m[iy0 - y0:iy1 - y0, ix0 - x0:ix1 - x0].astype(bool)).sum())
        return hit / n >= 0.6


    def add(self, cnt: np.ndarray, valid: np.ndarray, canvas_ppm: float, origin) -> None:
        """cnt: object outline in canvas px; valid: canvas mask of real pixels; origin: canvas px of
mm (0, 0)."""
        r = self._rasterize(cnt, valid, canvas_ppm, origin)
        if r is None:
            return
        x0, y0, x1, y1, obs_m, obj_m = r
        r_obs, r_obj = self.obs[y0:y1, x0:x1], self.obj[y0:y1, x0:x1]
        live = r_obs < self.freeze
        np.add(r_obs, obs_m, out=r_obs, where=live)
        np.add(r_obj, obj_m, out=r_obj, where=live)
        bx, by, bw, bh = cv2.boundingRect(obj_m)
        if bw and bh:
            nb = (x0 + bx, y0 + by, x0 + bx + bw, y0 + by + bh)
            self.bbox = nb if self.bbox is None else (min(self.bbox[0], nb[0]), min(self.bbox[1],
nb[1]),
                                                      max(self.bbox[2], nb[2]), max(self.bbox[3],
nb[3]))


    def to_mm(self, pts_px: np.ndarray) -> np.ndarray:
        return (np.asarray(pts_px, np.float64) - self.c) / self.ppm


    def measure(self, seed_mm) -> dict | None:
        if self.bbox is None:
            return None
        pad = 8
        x0, y0 = max(0, self.bbox[0] - pad), max(0, self.bbox[1] - pad)
        x1, y1 = min(self.G, self.bbox[2] + pad), min(self.G, self.bbox[3] + pad)
        obj, obs = self.obj[y0:y1, x0:x1].astype(np.uint16), self.obs[y0:y1, x0:x1]
        cons = ((obj * 2 > obs) & (obs > 0)).astype(np.uint8)
        cons = cv2.morphologyEx(cons, cv2.MORPH_OPEN, np.ones((3, 3), np.uint8))
        n, lab, stats, _ = cv2.connectedComponentsWithStats(cons, connectivity=8)
        if n < 2:
            return None
        sx = int(round(seed_mm[0] * self.ppm + self.c)) - x0 if seed_mm is not None else -1
        sy = int(round(seed_mm[1] * self.ppm + self.c)) - y0 if seed_mm is not None else -1
        if 0 <= sx < lab.shape[1] and 0 <= sy < lab.shape[0] and lab[sy, sx] > 0:
            idx = int(lab[sy, sx])
        else:
            idx = 1 + int(np.argmax(stats[1:, cv2.CC_STAT_AREA]))
        comp = (lab == idx).astype(np.uint8)
        self._cons = (x0, y0, x1, y1, comp)
        cnts, _ = cv2.findContours(comp, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
        cnt = max(cnts, key=cv2.contourArea)
        (cx, cy), (w, h), ang = cv2.minAreaRect(cnt)
        ring = cv2.dilate(comp, np.ones((5, 5), np.uint8)) & (1 - comp)
        is_open = bool(np.any((ring > 0) & (obs == 0)))
        box = cv2.boxPoints(((cx, cy), (w, h), ang)) + np.float32([x0, y0])
        poly = cv2.approxPolyDP(cnt, 0.006 * cv2.arcLength(cnt, True), True).reshape(-1, 2) +
np.int32([x0, y0])
        return {
            "length_mm": max(w, h) / self.ppm, "width_mm": min(w, h) / self.ppm,
            "area_mm2": float(comp.sum()) / (self.ppm ** 2), "open": is_open,
            "outline_mm": self.to_mm(poly), "box_mm": self.to_mm(box),
            "center_mm": self.to_mm(np.array([[cx + x0, cy + y0]]))[0],
            "_crop": (x0, y0, x1, y1, comp),
```

```
        }


    def seed_candidates(self, n: int = 24) -> list[tuple[float, float]]:
        """Deep-interior points of the object found so far (mm), spread along it, for re-seeding
when the
        original seed has left the camera's view."""
        m = self.measure(None)
        if m is None:
            return []
        x0, y0, x1, y1, comp = m["_crop"]
        d = cv2.distanceTransform(comp, cv2.DIST_L2, 3)
        if d.max() < 3:
            return []
        ys, xs = np.nonzero(d >= 0.6 * d.max())
        if len(xs) == 0:
            return []
        idx = np.linspace(0, len(xs) - 1, min(n, len(xs))).astype(int)
        order = np.lexsort((ys, xs))
        pts = np.column_stack([xs[order][idx] + x0, ys[order][idx] + y0])
        return [tuple(p) for p in self.to_mm(pts)]


    def preview_png_b64(self, m: dict, max_px: int = 240) -> str:
        """Small map: grey = seen, green = the object, black = never seen. For the sidebar."""
        x0, y0, x1, y1, comp = m["_crop"]
        pad = 40
        X0, Y0, X1, Y1 = max(0, x0 - pad), max(0, y0 - pad), min(self.G, x1 + pad), min(self.G, y1 +
pad)
        seen = self.obs[Y0:Y1, X0:X1] > 0
        img = np.zeros((Y1 - Y0, X1 - X0, 3), np.uint8)
        img[seen] = (70, 70, 70)
        sub = img[y0 - Y0:y1 - Y0, x0 - X0:x1 - X0]
        sub[comp > 0] = (80, 200, 80)
        s = min(1.0, max_px / max(img.shape[:2]))
        if s < 1:
            img = cv2.resize(img, None, fx=s, fy=s, interpolation=cv2.INTER_AREA)
        ok, buf = cv2.imencode(".png", img)
        return base64.b64encode(buf.tobytes()).decode() if ok else ""
```

calibrate.py

```
"""Camera calibration with a checkerboard (paper Section 4.1, Zhang's method).


From saved photos:
    python calibrate.py --images "calib/*.jpg"
Live from a webcam (SPACE = capture view, C = calibrate & save, Q = quit):
    python calibrate.py --camera 0


Capture 15-30 views with the board tilted (pitch/roll/yaw), at different distances,
and covering the corners of the frame -- corner coverage is what pins down distortion.
"""
import argparse
import glob


import cv2
import numpy as np


CRITERIA = (cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER, 30, 1e-3)



def find_corners(gray, pattern):
    ok, corners = cv2.findChessboardCorners(gray, pattern, cv2.CALIB_CB_ADAPTIVE_THRESH +
cv2.CALIB_CB_NORMALIZE_IMAGE)
    if ok:
        corners = cv2.cornerSubPix(gray, corners, (11, 11), (-1, -1), CRITERIA)
    return ok, corners



def calibrate(image_points, image_size, pattern, square_mm, out):
    objp = np.zeros((pattern[0] * pattern[1], 3), np.float32)
    objp[:, :2] = np.mgrid[0 : pattern[0], 0 : pattern[1]].T.reshape(-1, 2) * square_mm
    rms, K, dist, _, _ = cv2.calibrateCamera([objp] * len(image_points), image_points, image_size,
None, None)
    dist = dist.ravel()
    np.savez(out, camera_matrix=K, dist_coeffs=dist, image_size=np.array(image_size), rms=rms)
    print(f"\nViews used: {len(image_points)}   image size: {image_size[0]}x{image_size[1]}")
    print(f"RMS reprojection error: {rms:.3f} px   (aim for < 0.5 px)")
    print(f"fx = {K[0, 0]:.2f} px, fy = {K[1, 1]:.2f} px")
    print(f"cx = {K[0, 2]:.2f} px, cy = {K[1, 2]:.2f} px")
    print("Distortion: k1={:.4f}, k2={:.4f}, p1={:.4f}, p2={:.4f}, k3={:.4f}".format(*dist[:5]))
    print(f"Saved -> {out}")



def from_images(pattern_glob, pattern):
    points, size = [], None
    for path in sorted(glob.glob(pattern_glob)):
        gray = cv2.imread(path, cv2.IMREAD_GRAYSCALE)
        if gray is None:
            continue
        size = gray.shape[::-1]
        ok, corners = find_corners(gray, pattern)
        print(f"{'ok  ' if ok else 'MISS'} {path}")
        if ok:
            points.append(corners)
    return points, size



def from_camera(index, pattern, width, height):
    cap = cv2.VideoCapture(index)
    if width:
        cap.set(cv2.CAP_PROP_FRAME_WIDTH, width)
        cap.set(cv2.CAP_PROP_FRAME_HEIGHT, height)
    points, size = [], None
    while True:
        ok, frame = cap.read()
        if not ok:
            break
        gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
        size = gray.shape[::-1]
        found, corners = cv2.findChessboardCorners(gray, pattern, cv2.CALIB_CB_FAST_CHECK)
        view = frame.copy()
        cv2.drawChessboardCorners(view, pattern, corners, found)
        cv2.putText(view, f"captured: {len(points)}  SPACE=capture  C=calibrate  Q=quit", (10, 30),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 0) if found else (0, 0, 255), 2)
        cv2.imshow("calibrate", view)
```

```
        key = cv2.waitKey(1) & 0xFF
        if key == ord(" ") and found:
            ok2, refined = find_corners(gray, pattern)
            if ok2:
                points.append(refined)
                cv2.imwrite(f"calib_{len(points):02d}.png", frame)
        elif key == ord("c") and len(points) >= 10:
            break
        elif key == ord("q"):
            points = []
            break
    cap.release()
    cv2.destroyAllWindows()
    return points, size



if __name__ == "__main__":
    ap = argparse.ArgumentParser(description=__doc__,
formatter_class=argparse.RawDescriptionHelpFormatter)
    src = ap.add_mutually_exclusive_group(required=True)
    src.add_argument("--images", help='glob, e.g. "calib/*.jpg"')
    src.add_argument("--camera", type=int, help="webcam index")
    ap.add_argument("--cols", type=int, default=9, help="inner corners per row")
    ap.add_argument("--rows", type=int, default=6, help="inner corners per column")
    ap.add_argument("--square-mm", type=float, default=25.0)
    ap.add_argument("--width", type=int, default=1920)
    ap.add_argument("--height", type=int, default=1080)
    ap.add_argument("--out", default="calibration.npz")
    a = ap.parse_args()


    pattern = (a.cols, a.rows)
    if a.images:
        pts, size = from_images(a.images, pattern)
    else:
        pts, size = from_camera(a.camera, pattern, a.width, a.height)
    if len(pts) < 10:
        raise SystemExit(f"Need at least 10 good views, got {len(pts)}.")
    calibrate(pts, size, pattern, a.square_mm, a.out)
```

Appendix B: Additional Output Screenshots

Figure B.1 to Figure B.3 show the intermediate images of the pipeline for the 45° scene of Figure 6.1, and Figure B.4 the frontal case.

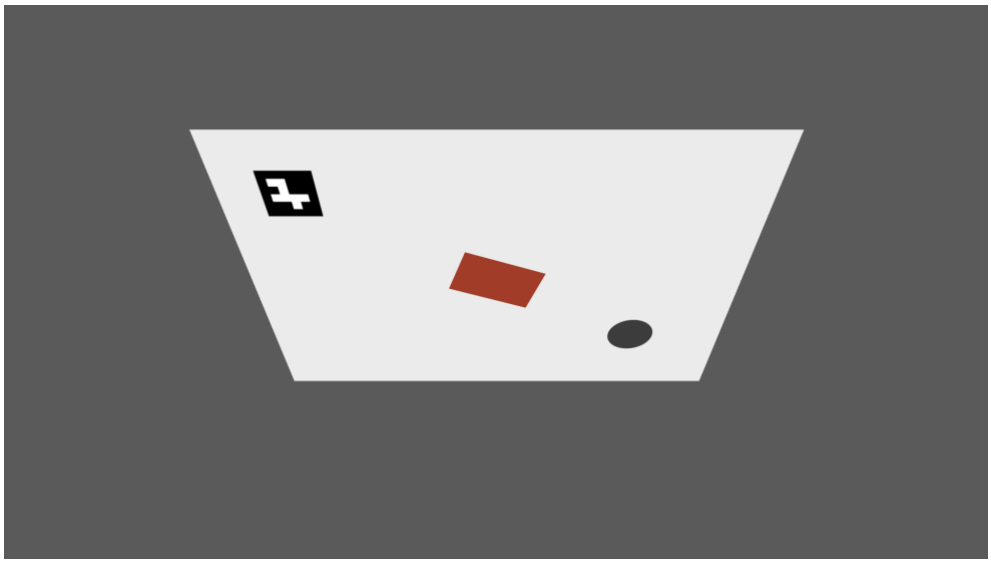


Figure B.1. Input frame, plane tilted 45° (rendered scene).

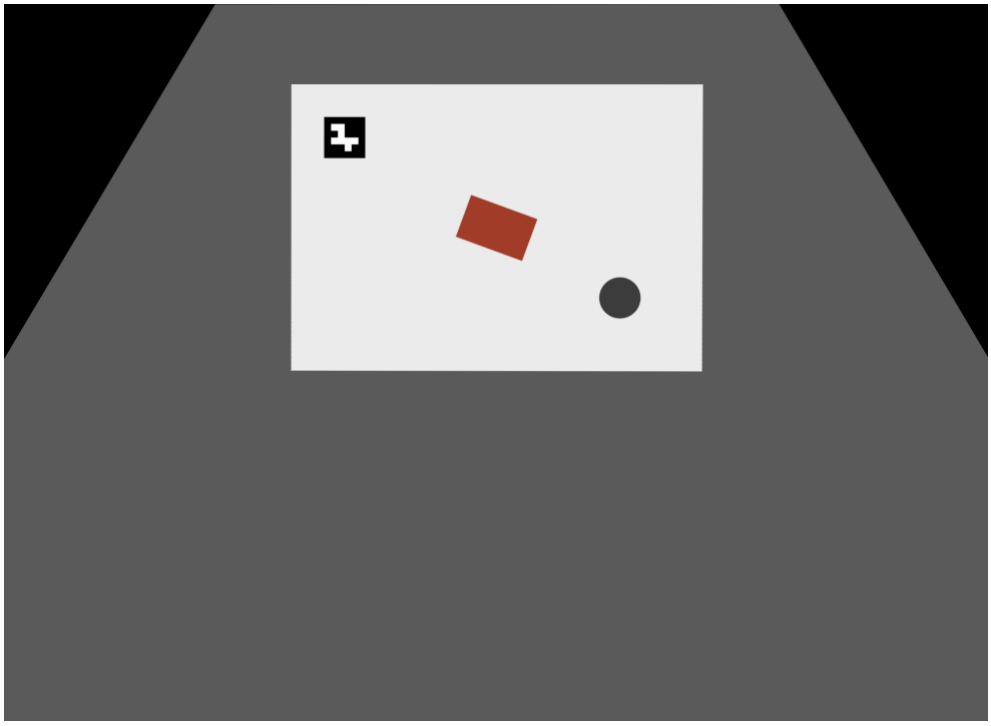


Figure B.2. Rectified top-down view produced by the homography: the marker is square again.

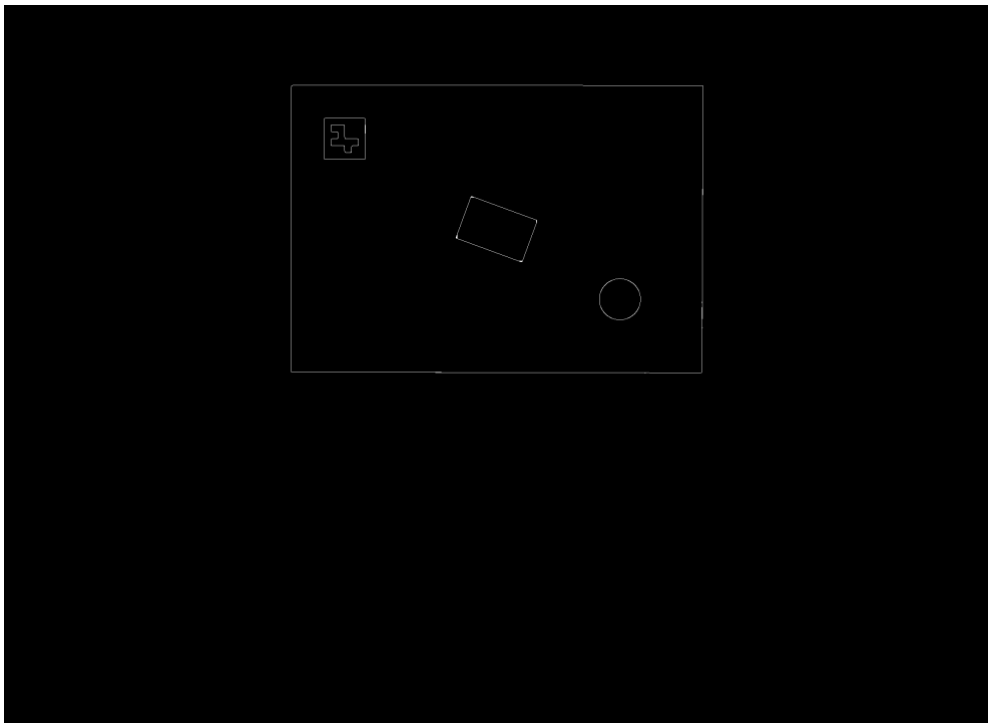


Figure B.3. Edge map of the rectified view after CLAHE, Canny and closing.

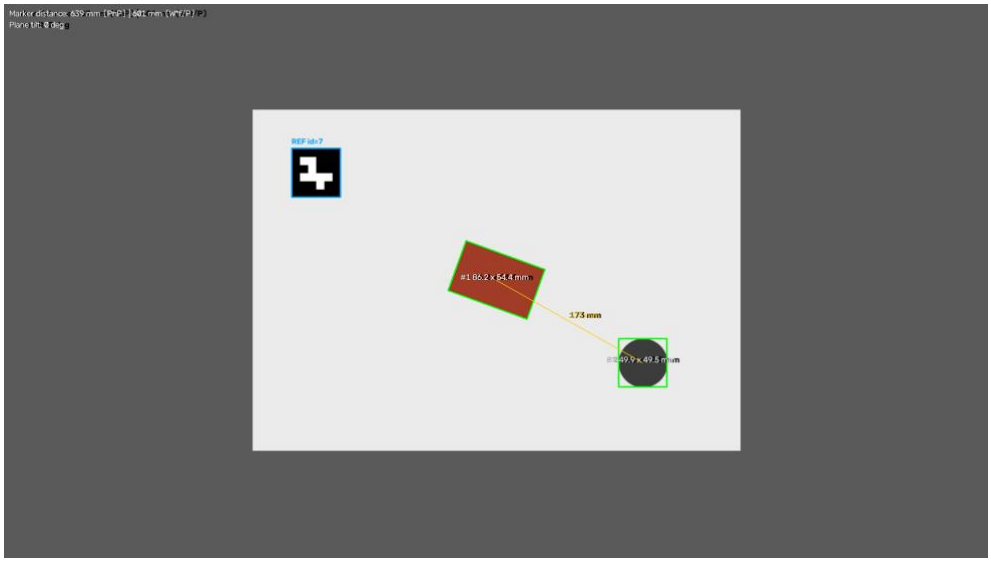


Figure B.4. Output with the camera facing the plane: card 86.2 × 54.4 mm, disc 49.9 × 49.5 mm, distance 639 mm (true 637 mm).

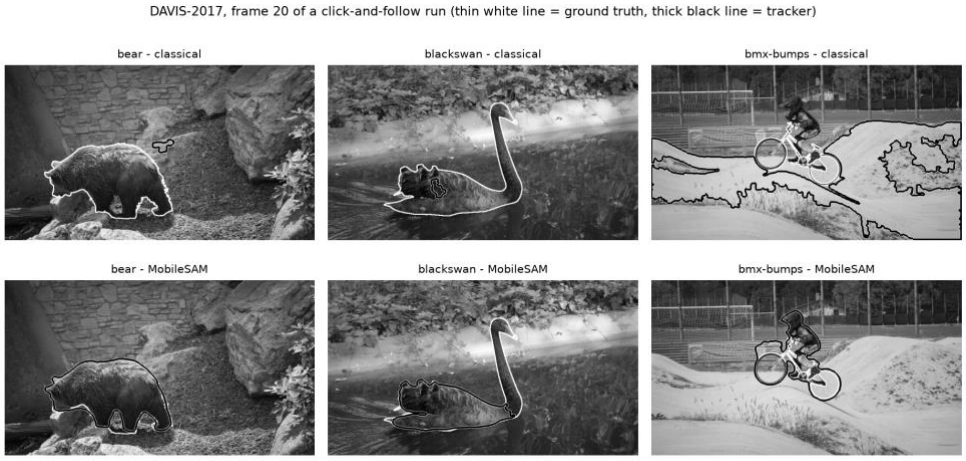


Figure B.5. Frame 20 of three DAVIS sequences. Top: classical outline. Bottom: MobileSAM (real video).

Appendix C: GitHub Repository Link

Repository: **[add the GitHub repository URL here]**. It contains the Python library, the macOS application, the tests, the evaluation scripts and the README with the run instructions of Section 5.4.

Appendix D: Video Demonstration Link

Video: **[add the video link here]** (file `Spatial_Measure_Demo.mp4`, 1080p, about 8.5 minutes). It explains the working of the program, the logic of each stage with the code, and the output on the 0°, 25° and 45° scenes and the automated tests.